

Dateisysteme

Andreas Scheibenpflug, MSc
SE Bachelor (BB/VZ), 2. Semester

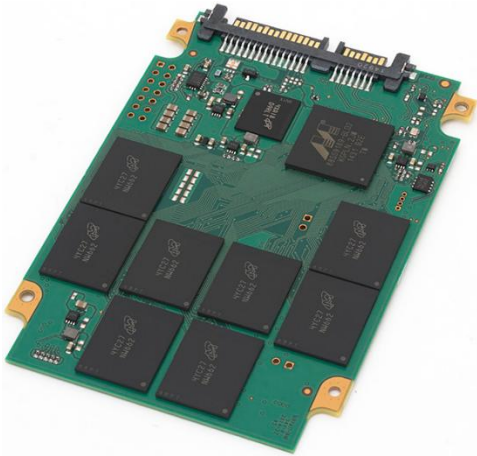
Übersicht

- Festplatten: Allgemeines
 - Diskgeometrie HDD & SSD
- Dateisysteme (logische Sicht)
 - Aufgabe
 - Dateien vs. Verzeichnisse
 - Dateiverwaltung
- Partitionierung und Booten eines Computers
- Dateisysteme (aus OS/Implementierungssicht)
 - Adressierung
 - Layout eines generischen Dateisystems
 - Implementierung von Dateisystemen
 - Beispiele von konkreten Dateisystemen (ext4)
 - Erweiterte Funktionalitäten

Festplatten: Allgemeines

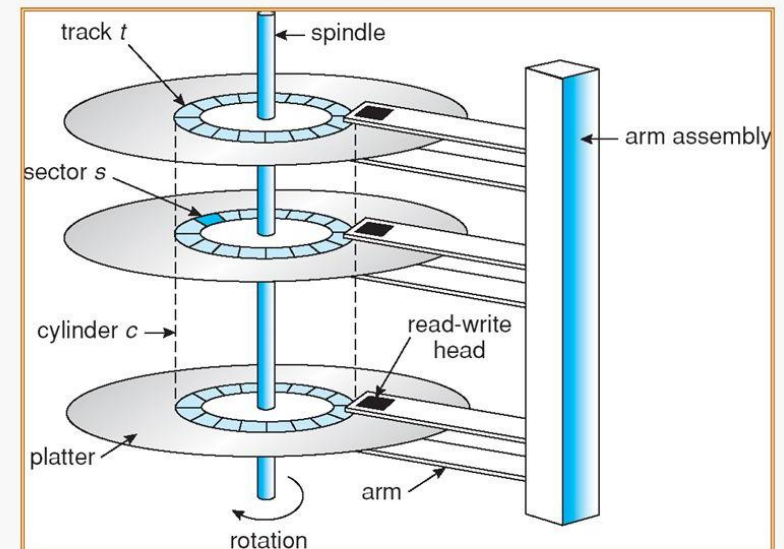


Allgemeines - Festplatten



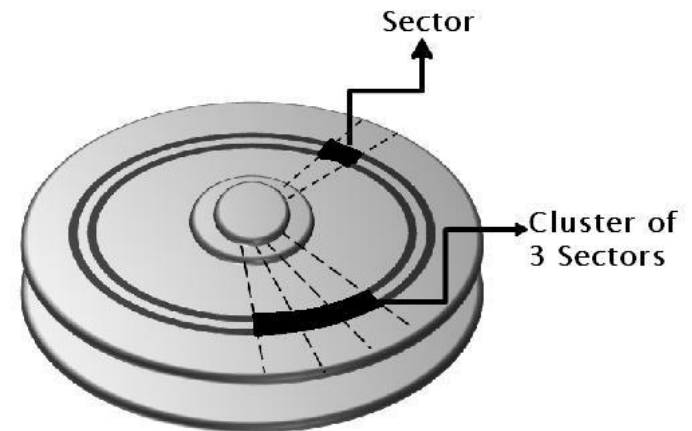
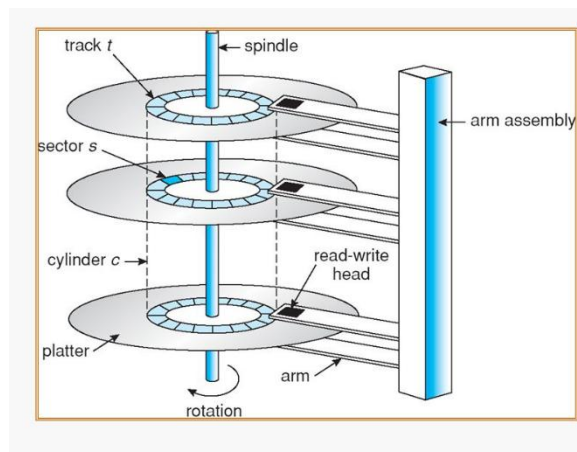
Diskgeometrie HDD

- Mechanisches, magnetisches Speichermedium
- Daten werden auf die rotierenden Scheiben geschrieben
- Besteht aus Sektoren, Zylindern, Sektoradressen, Schreib-Lese-Kopf
- Entwickler IBM
- Vorgänger: Magnetband (siehe Geschichte)



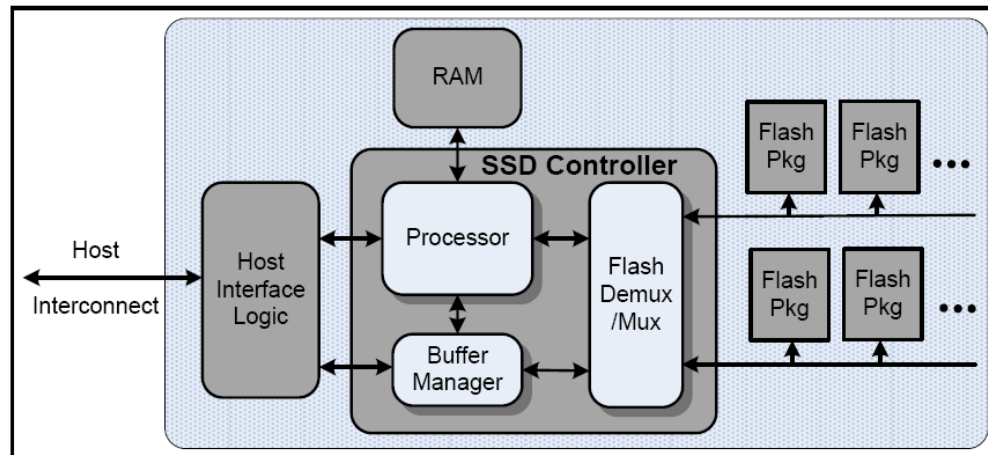
Blöcke / Cluster an HDD

- Mehrere physikalisch aufeinander folgende Sektoren werden zu sogenannten Clustern (Windows) oder Blöcken (Unix) zusammengefasst
- Dadurch können beim Datentransfer die Bewegungen der Lese- / Schreibköpfe reduziert werden

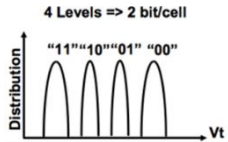


Diskgeometrie SSD

- Speichermedium das wie eine herkömmliche magnetische Festplatte eingebaut und angesprochen werden kann
- Keine rotierenden Scheiben
- Besteht aus Controllerchip, Flash-Bausteinen, Cache
- Flash-Bausteine beinhalten Speicherzellen (NAND-Flash Zellen)



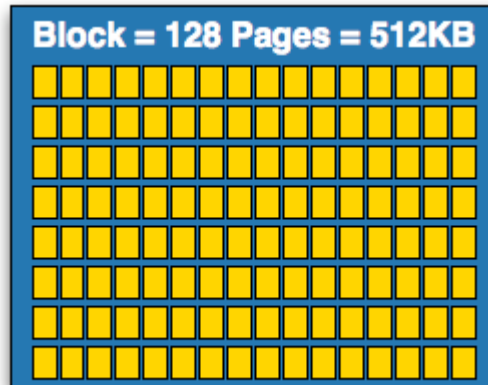
Speichereinheiten SSD - Beispiel



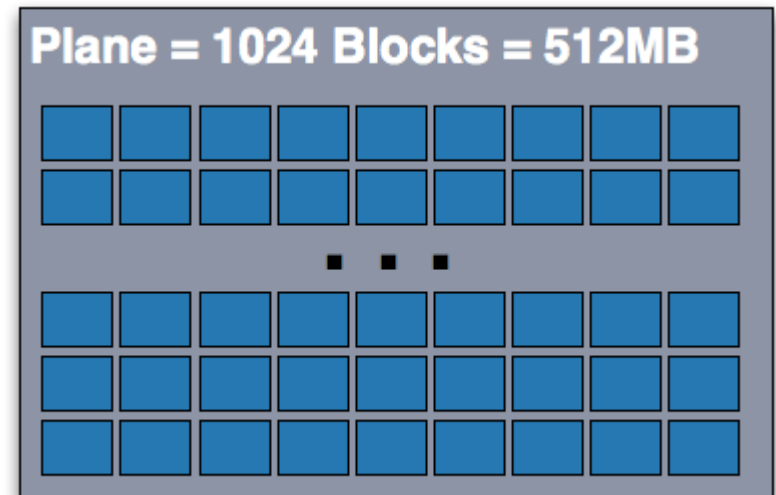
NAND-Flash Zellen werden zu einer Page (4K) gruppiert



Pages werden zu einem Block (512K) gruppiert

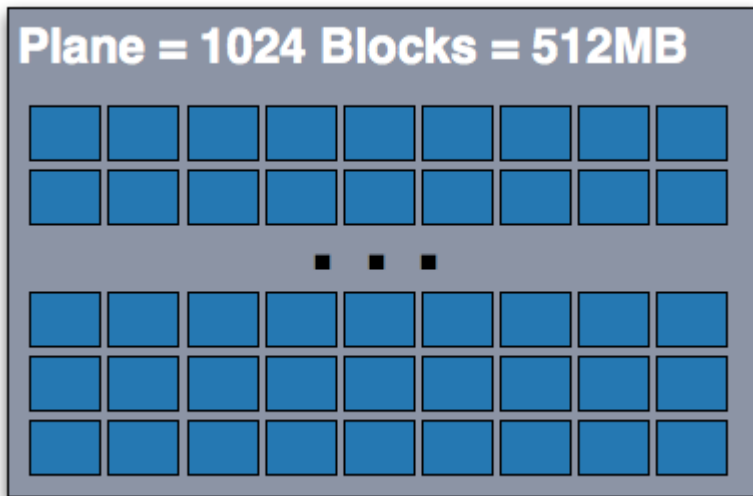


Blocks werden zu einer Plane gruppiert

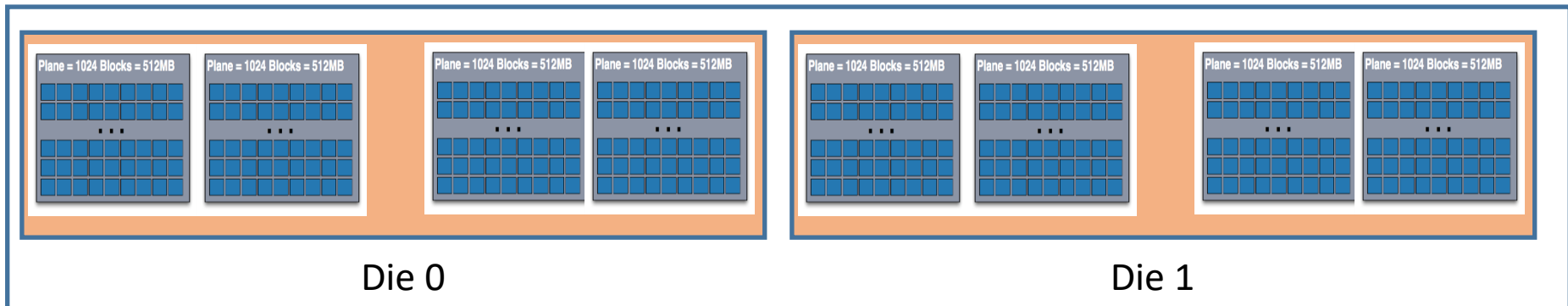


Speichereinheiten SSD

Arrays of Blocks werden
zu einer Plane gruppiert



- Je vier Planes werden zu einem Die gruppiert
- Jedes Die hat 2 GB Speicher
- Zwei Dies ergeben ein 4 GB Flash Package



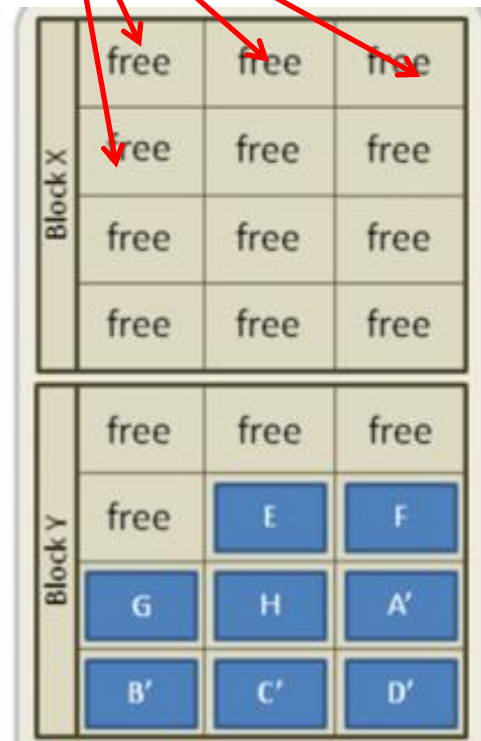
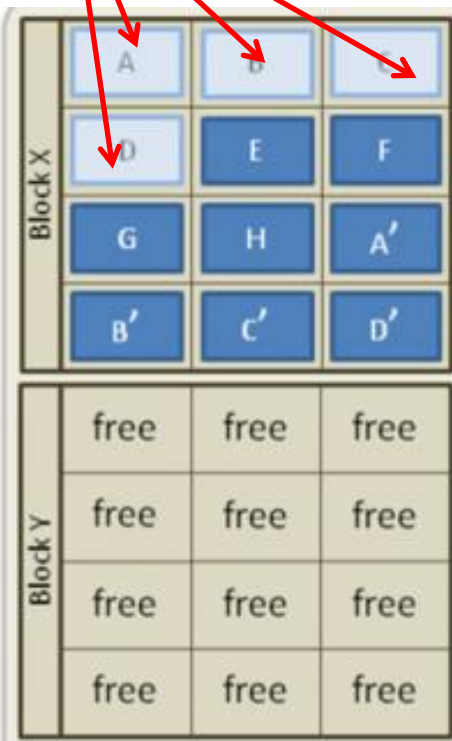
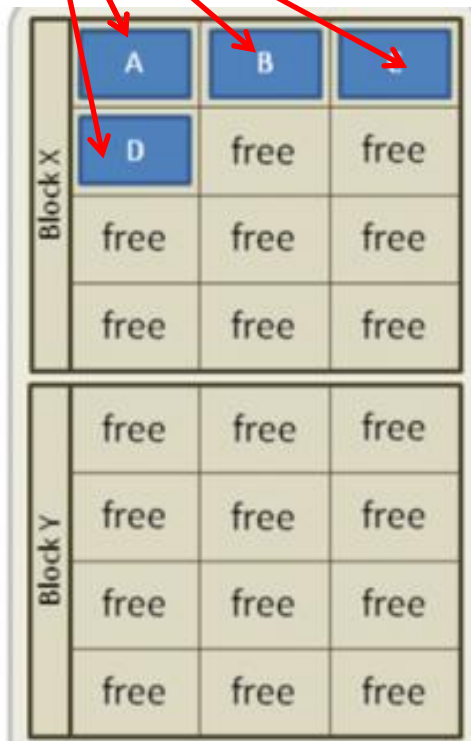
SSD: Zustände Pages

Pages können nur geschrieben werden
Nur ganze Blöcke können gelöscht werden

live pages

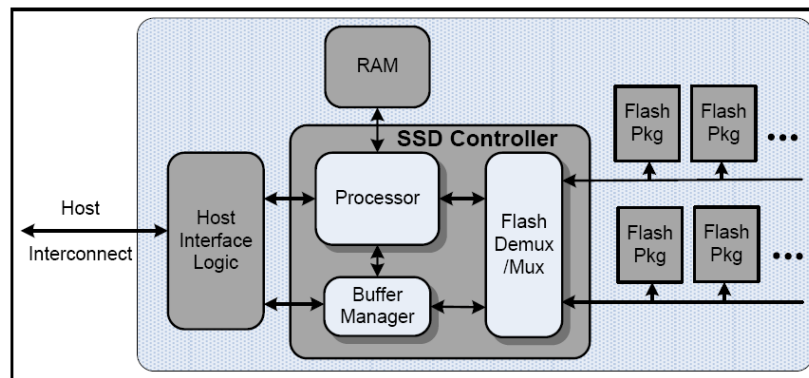
dead pages

free pages



Diskgeometrie SSD

- SSD soll gleichmäßig „altern“
- SSD Controller verwendet Wear Levelling:
 - Umorganisieren der Pages
 - Neu ankommende Daten werden in die bisher am wenigsten beanspruchten Blöcke geschrieben
 - Controller verwendet Tabellen:
 - zur Zuordnung der Blockadressen (LBA), die das Dateisystem verwendet
 - zur Zuordnung der tatsächlichen Pageadressen im Flash Speicher (Dateisystem hat keine Ahnung mehr, wo Daten tatsächlich gespeichert sind)



Dateisysteme - logische Sicht

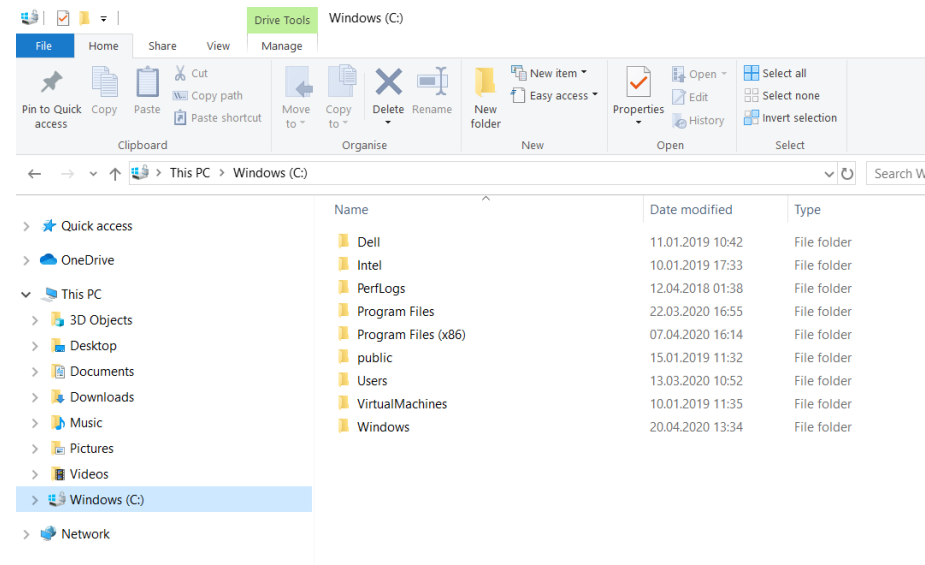
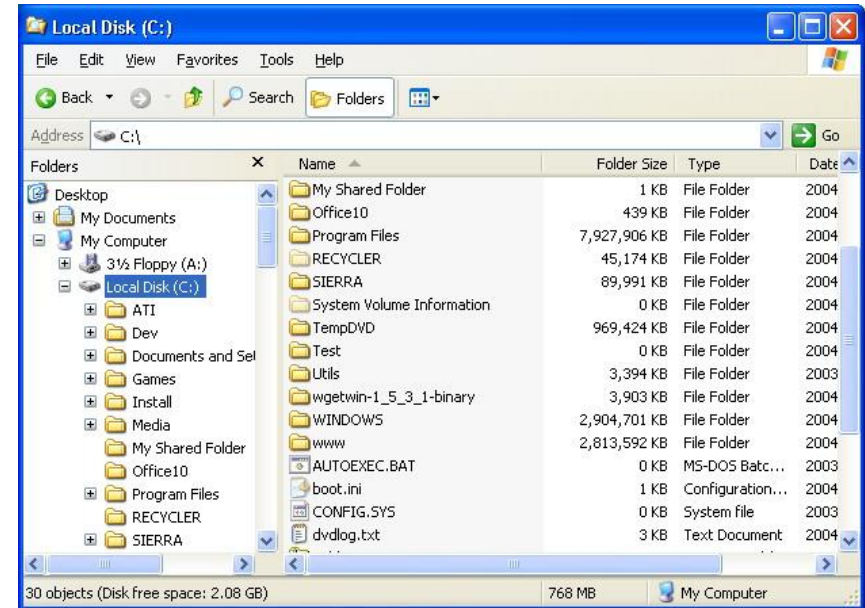
Was ist ein Dateisystem?

- Schnittstelle zwischen dem Betriebssystem und den Partitionen auf den Datenträgern
- Ablageorganisation
- Aufgabe: **Dateien** sollen gespeichert werden können, sollen leicht wieder gefunden und gelesen werden können

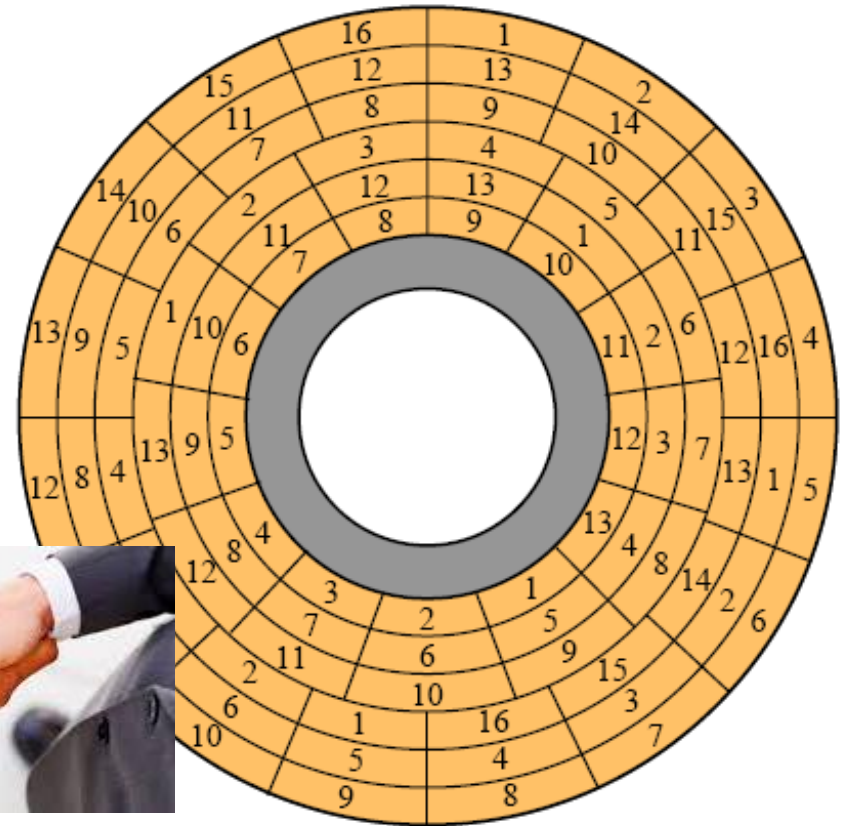
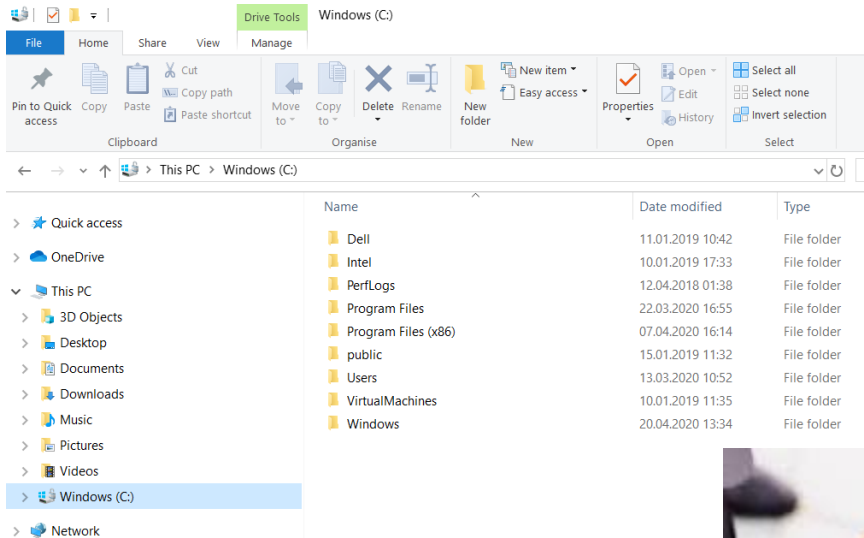
Dateisystem



- Nach der Einteilung einer Festplatte in Partitionen muss jede Partition mit einem Dateisystem formatiert werden, bevor sie zur Datenspeicherung benutzt werden kann
- Dateisystem kann sich auf eine Verzeichnisstruktur beziehen oder auf Partitionen
- Dateisysteme benutzen Datenstrukturen und Funktionen zur persistenten Speicherung der Daten



Physische Sicht des Dateisystems

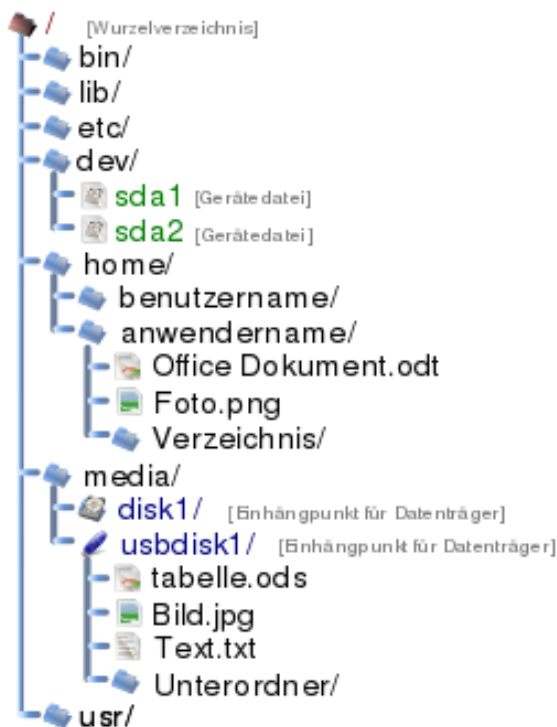


Aufgabe eines Dateisystems

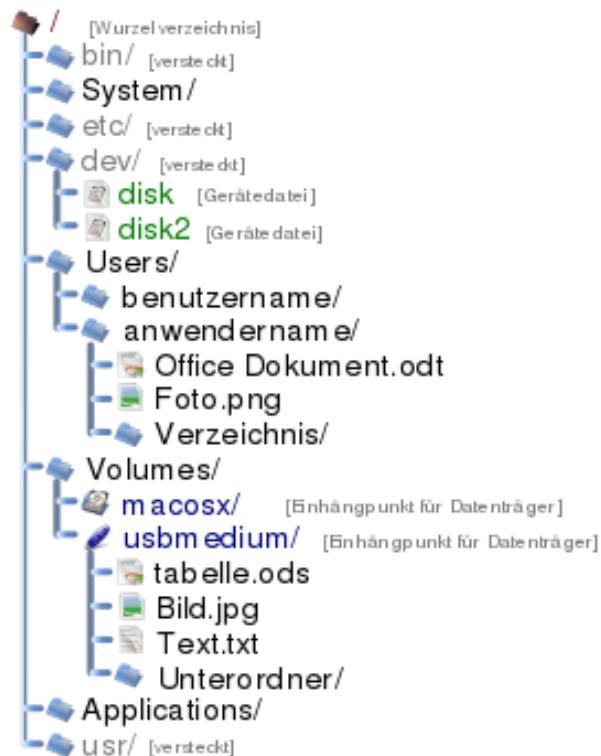
- Dateisystem ist das vielleicht am meisten sichtbare Konzept eines Betriebssystems (siehe Explorer)
- Es verwendet folgende Objekte:
 - Dateien
 - Verzeichnisse zur Strukturierung von Dateien
 - Laufwerke für Partitionen (Windows)
- Persistente Speicherung großer Informationsmengen
- Gleichzeitiger Zugriff auf Daten durch mehrere Prozesse (siehe Prozessmanagement) muss möglich und geregelt sein
- Berechtigungskonzept für den Zugriff auf Daten ist notwendig

Logische Sicht des Dateisystems

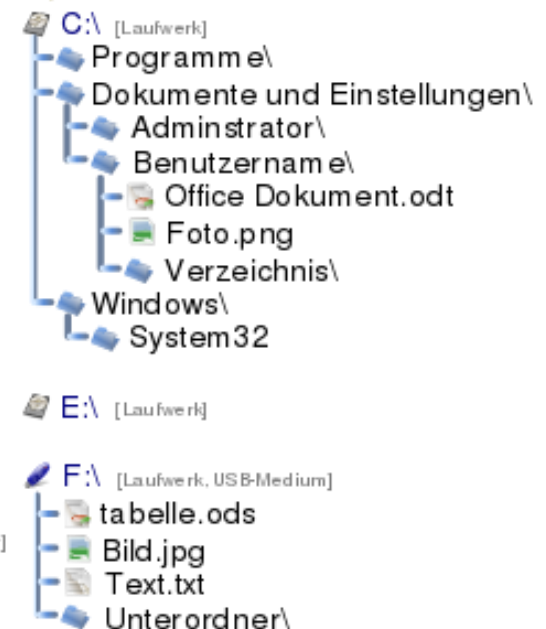
Linux, unixoide, ZETA



Mac OS X

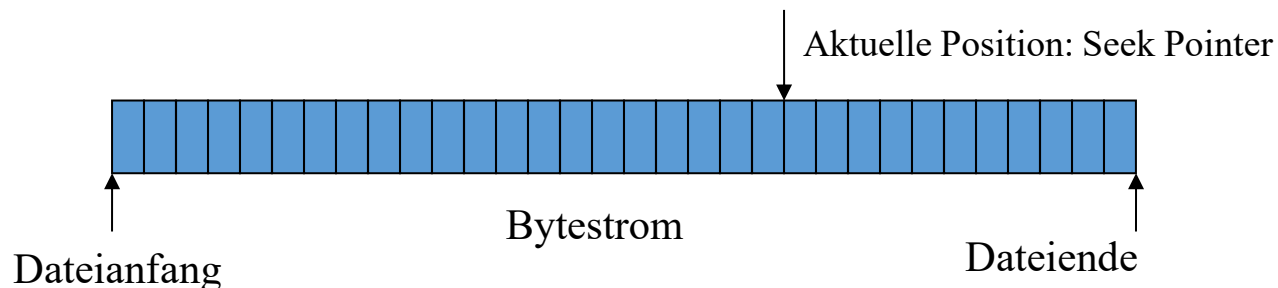


Windows NT/2000/XP



Logische Sicht: Datei

- Jede Datei besteht aus:
 - Dateiattribute (Metadaten): Datum, Zeitpunkt, Dateiname, Dateierweiterung (Windows: Zuordnung Programm, mit dem die Datei zu bearbeiten ist; Unix: Endung nur eine Konvention), Dateigröße (in Bytes), Benutzer- und Zugriffsinformationen, Markierungsbits oder Flags
 - Nutzdaten: werden als Bytestrom gespeichert
 - Struktur des Bytestroms selbst ist für das Dateisystem nicht relevant, sondern nur für die Anwendung, die die Nutzdaten organisiert
 - Dateizugriff: sequenziell (früher), zufällig (random access)

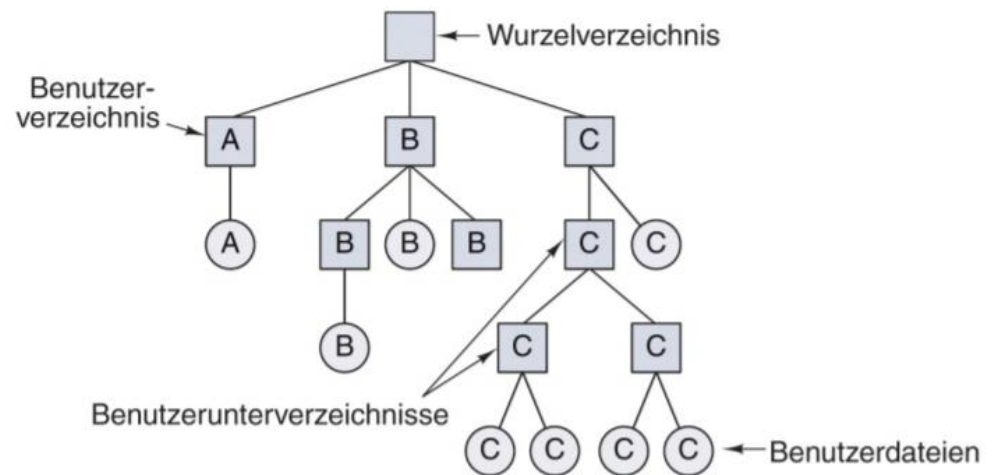


Dateitypen

- Reguläre Dateien (regular file)
- Verzeichnisse (directory)
- Zeichendateien (character special file)
 - Device File für direkte, serielle Kommunikation mit Geräten (meist nicht gecached), z.B. USB
- Blockdateien (special block file)
 - Device File für blockorientierte Geräte (Daten werden im OS gecached und in Blöcken (parallel) übertragen), z.B. Festplatten
- Dateien existieren, um Informationen zu speichern, die später wieder abgerufen werden. Dafür stehen unterschiedliche Operationen für Speicherung und Abfrage zur Verfügung!

Logische Sicht: Verzeichnisse

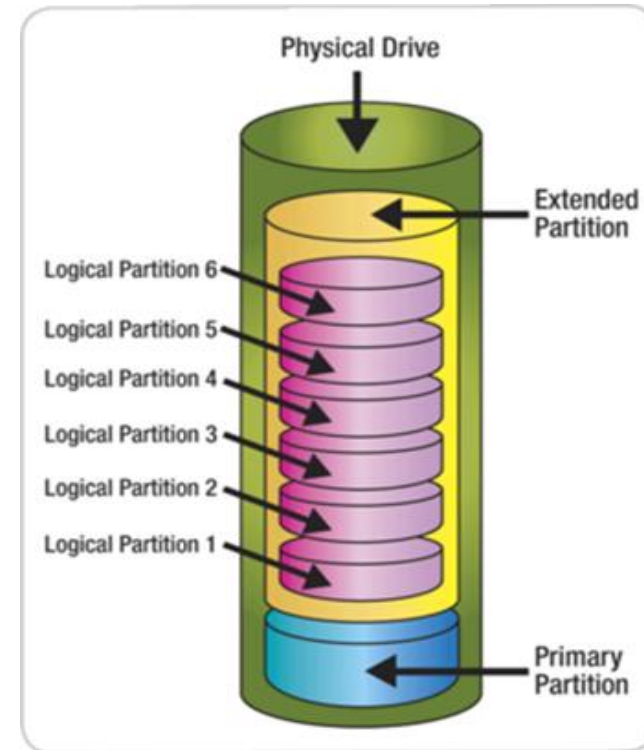
- Verzeichnisse werden verwendet zur thematischen Gruppierung von Dateien durch den Benutzer oder dem Betriebssystem
- Hierarchische Verzeichnissysteme
- Verzeichnisse können Unterverzeichnisse enthalten, dadurch entsteht eine Baumstruktur
- Verzeichnisse werden von verschiedenen Dateisystemen mit unterschiedlichen Datenstrukturen implementiert



Partitionierung und Booten eines Computers

Partitionen und Partitionstypen

- Partitionen:
 - Einteilung einer Harddisk in einen oder mehrere Bereiche hintereinanderliegender Sektoren
- Partitionstypen (erfolgt mithilfe eines Partitionierungsprogramms)
 - Primäre Partition:
 - Kann nicht mehr weiter unterteilt werden
 - Von diesen kann gebootet werden
 - Erweiterte Partition:
 - Kann alleine nicht benutzt werden
 - Unterteilung in logische Laufwerke
- BIOS/MBR: Maximal 4 primäre Partitionen oder 3 primäre und 1 erweiterte Partition sind möglich
- UEFI/GPT: 128 primäre Partitionen möglich → Keine erweiterten Partitionen mehr wirklich benötigt

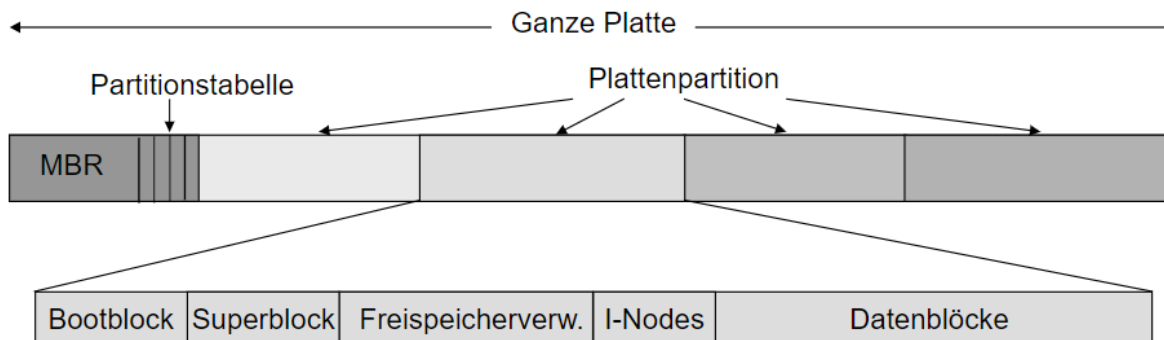


Booten eines Computers

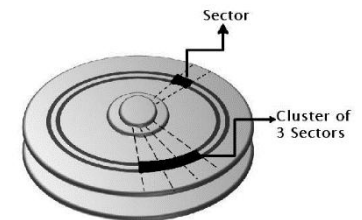
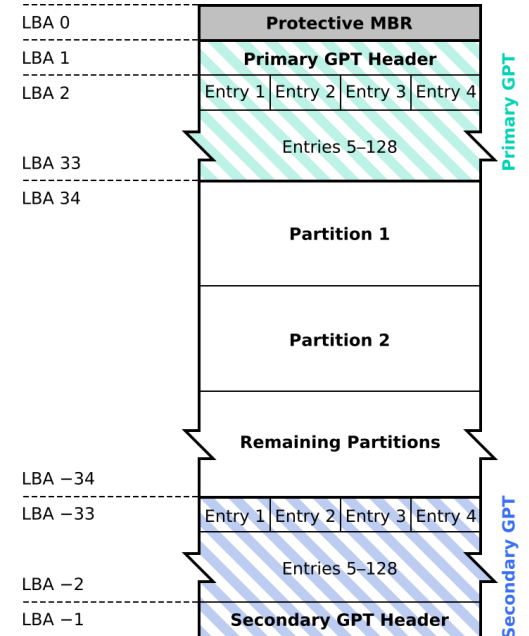
- Hochfahren ist ein Prozess bestehend aus mehreren Schritten = Booten
- PC: besitzt Motherboard und darauf Programm: Boot-ROM
- Boot-ROM: Programm (Firmware), enthält Ein-/Ausgabesoftware, Initialisiert Hardware, Stellt Services für OS bereit
- Boot-ROM bestimmt das Gerät, von dem das Betriebssystem geladen werden soll
- Bekannte Boot-ROM Implementierungen:
 - Basic Input/Output System (BIOS, veraltet)
 - Unified Extensible Firmware Interface (UEFI)
- BIOS lädt Master Boot Record (MBR)
- UEFI lädt GUID Partition Table (GPT)

Layout Festplatte, Partitionen, Dateisysteme

- Festplatte:
 - Sektor 0 reserviert für MBR
 - Sektor 1..x (min. 16 KB) reserviert für GPT
- Header zeigt auf die Partitionstabelle
 - Enthält die Start-/Endadressen der Partitionen und zusätzliche Informationen



GUID Partition Table Scheme



Booten eines Computers

- UEFI lokalisiert Boot Gerät, liest dessen GPT und bestimmt die EFI system partition (ESP)
- ESP enthält den Boot Loader, welcher von UEFI geladen wird
 - Zu diesem Zeitpunkt wird die Kontrolle von UEFI an den Boot Loader übergeben
- Boot Loader ist dafür verantwortlich, ein OS von einer Partition zu laden
 - Boot Loader kann einen Auswahldialog bieten, enthält Geräte- und Dateisystemtreiber
- Boot Loader lädt das OS von einer Partition und übergibt die Kontrolle an OS
- OS fragt UEFI nach Konfigurationseinstellungen und bestimmt Geräte
- Für jedes gefundene Gerät wird nach Gerätetreibern gesucht
- Wenn alle Treiber gefunden wurden, lädt das OS diese in den Kern, initialisiert HW
- Danach: Interne Tabellen initialisieren, nötige Hintergrundprozesse starten (init!), Login-Shell bzw. GUI starten

Dateisysteme - OS/Implementierungssicht

Ziele Dateisystem: Partitionsverwaltung

- Die Partition ist aus Sicht des DS so zu verwalten, dass
 - die Positionierungszeit der Lese-/Schreibköpfe minimal ist (besonders bei mechanischen Festplatten wichtig)
 - d.h. dass alle Blöcke einer Datei möglichst nahe beieinander liegen sollten (Fragmentierungsproblem)
 - die vorhandene Diskkapazität bestmöglich ausgenutzt wird (Blockungsfaktor)

Dateiverwaltung als Service des OS

- Folgende Operationen werden zur Verfügung gestellt (vom System Call Interface des OS):
 - Datei Funktionen: create, delete, open, close, read, write, append, seek, rename
 - Verzeichnis Funktionen: create, delete, opendir, closedir, rename
- Verwaltungsfunktionen:
 - Formatierung: Erzeugen eines leeren Dateisystems auf einer Partition
 - Konsistenzprüfung beim Rechnerstart
 - Backup Funktionen für Datensicherung
 - Falls gewünscht: Komprimierung und Verschlüsselung der Dateien

Partitionen und Adressierung

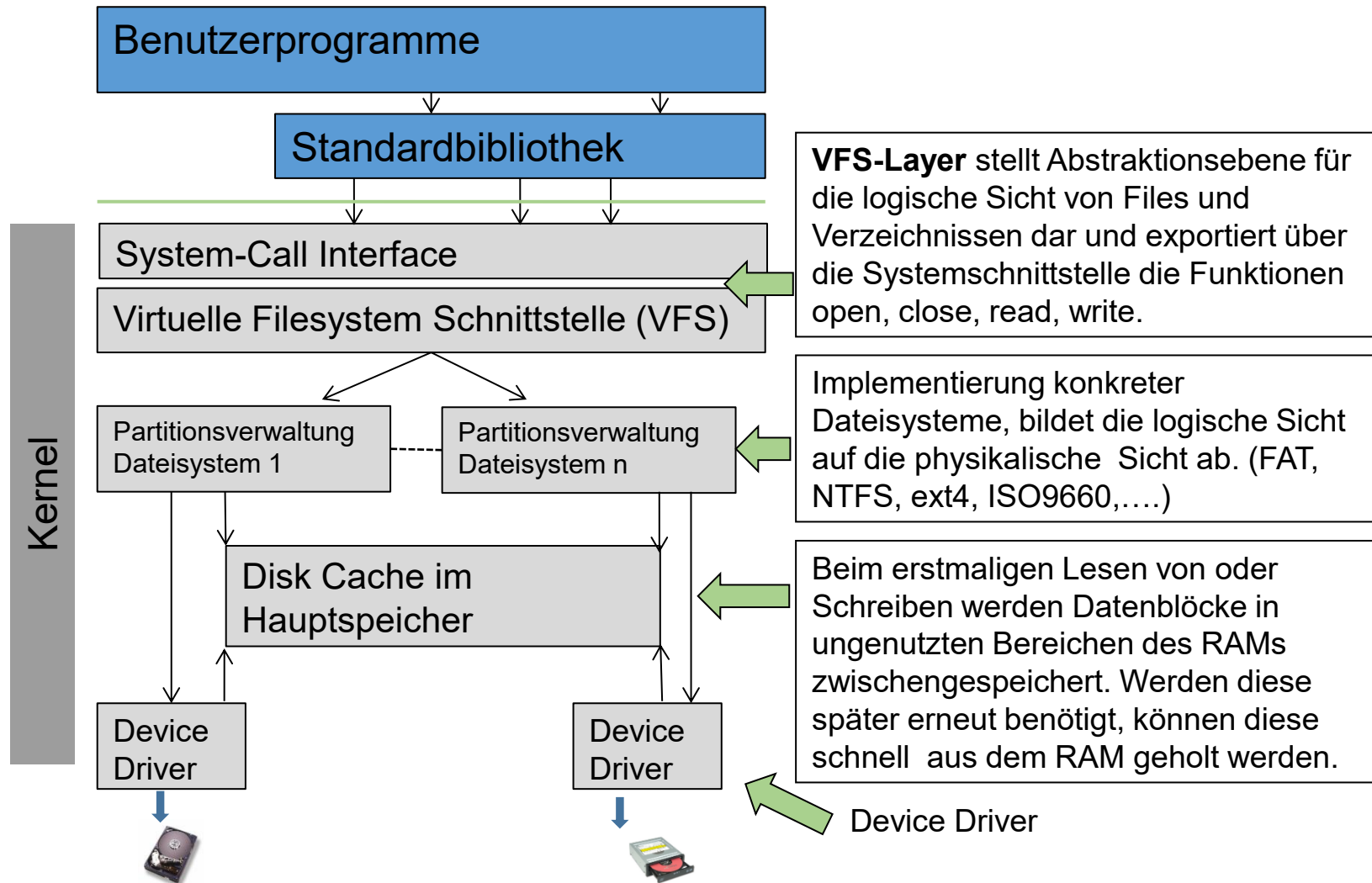
- Eine Partition stellt für ein Dateisystem ein Array von fortlaufend nummerierten Blöcken dar, die frei oder belegt sein können
- Jeder Block kann direkt über seine Logical Block Address (LBA) adressiert werden
- Das Dateisystem muss seine eigene Verwaltungsinformation auf der Partition speichern
- Das Dateisystem sieht eine Datei/Verzeichnis als Folge von Blöcken, die physikalisch nicht aufeinander folgen müssen



Adressierung

- LBAs sind Adressen, mit welchem das Dateisystem/OS arbeitet
- LBAs werden vom Festplatten Controller in physische Adressen übersetzt
- Vorteil: Benötigt kein Wissen wie das darunterliegende Speichergerät technisch funktioniert
 - z.B. alte Adressierungsschema für HDDs war Cylinder-head-sector (CHS) Adressierung
 - Macht nur für HDDs Sinn
- Standard für HDDs/SSDs
- LBA-Adressen haben aktuell eine Länge von 48 Bit
 - Gestartet wurde mit 22 Bit

Schichten des Dateisystems im Kernel



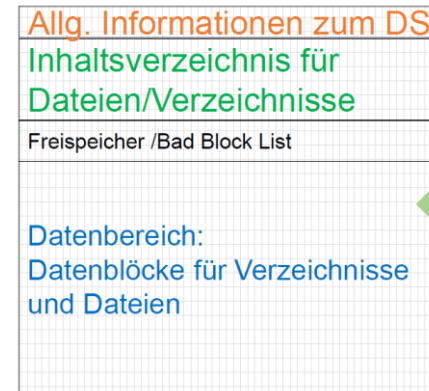
Layout eines generischen Dateisystems



Für alle Bereiche sind entsprechende Daten-Strukturen vorzusehen, die über die Sektoren der Partition zu legen sind

Partition mit Sektoren

Generisches Dateisystem



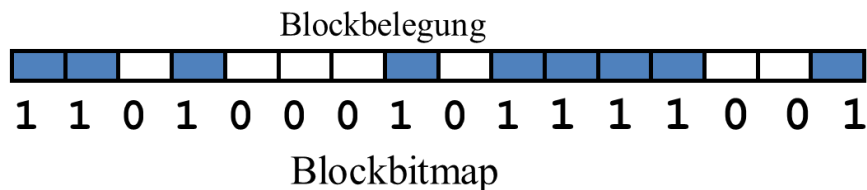
Für alle Bereiche sind entsprechende Daten-Strukturen vorzusehen, die über die Sektoren der Partition zu legen sind.

Partition mit Sektoren

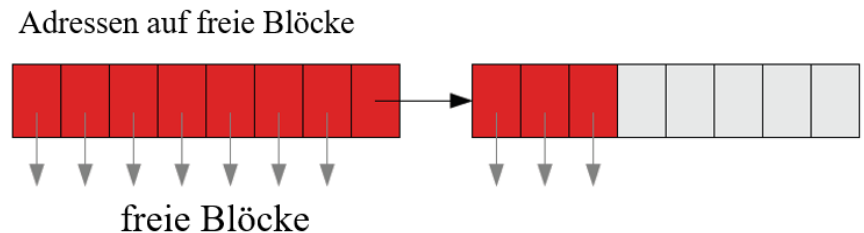
- Allgemeine Informationen des Dateisystems:
 - Kennung des spezifischen Dateisystems, Größe in Blöcken, Dirty-Bit: Indikator, ob eine Überprüfung oder ggf. Reparatur beim Systemstart durchzuführen ist
 - Freie Positionen im Inhaltsverzeichnis
- Inhaltsverzeichnis für Dateien/Verzeichnisse
 - Beinhaltet für jede Datei/Verzeichnis eine Datenstruktur, die Dateiattribute und Referenzen auf Datenblöcke beinhaltet
- Freispeicherverwaltung, Bad-Blocks Verwaltung
- Datenbereich
 - Beinhaltet die eigentlichen Datenblöcke (= Nutzdaten) für Dateien/Verzeichnisse und die leeren Datenblöcke

Freispeicherverwaltung

- Ziel: Welche Plattenblöcke sind mit welchen Dateien assoziiert
- 2 Möglichkeiten:
 - Bitmaps
 - Verkettete Listen



1 – Block/Cluster ist belegt, oder BAD
0 – Block/Cluster ist noch frei

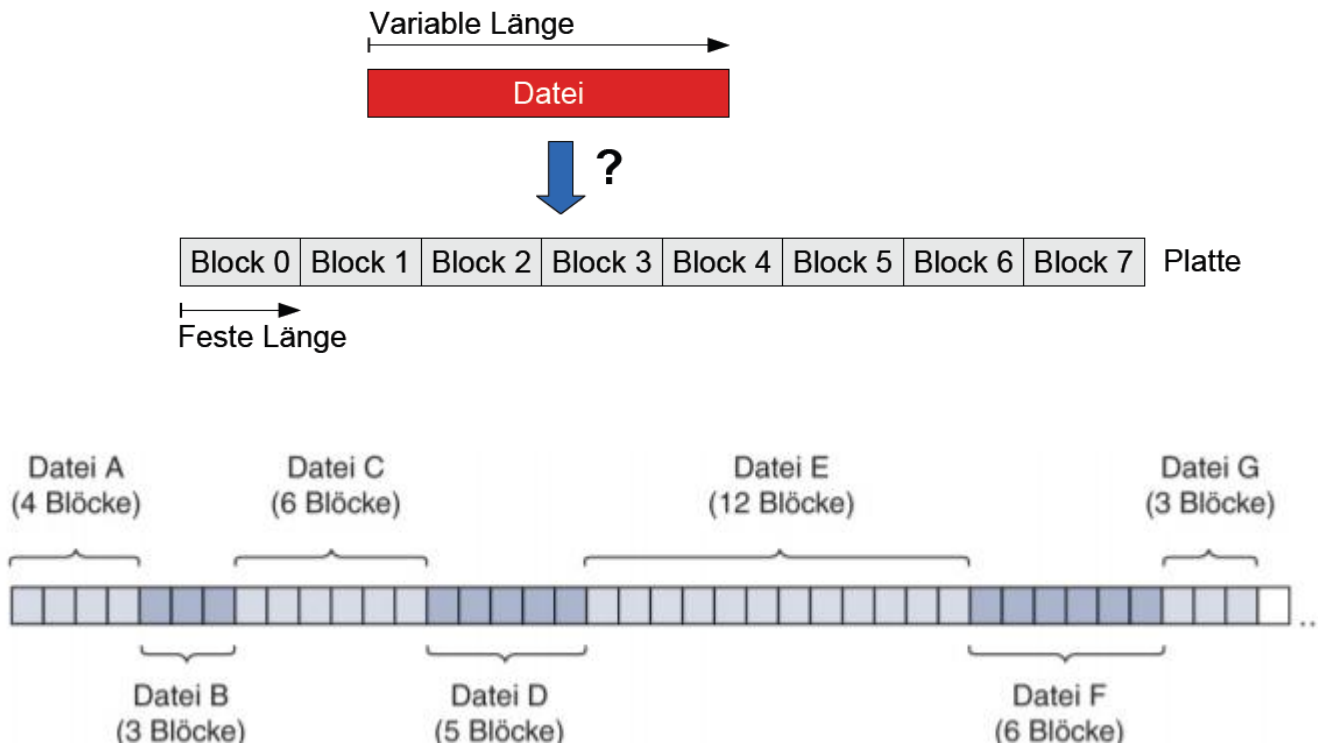


Liste enthält Einträge auf freie Blöcke
Rot – freie Blöcke
Grau – BAD Block (kaputt), oder schon belegt mit Daten

Inhaltsverzeichnis für Dateien/Verzeichnisse

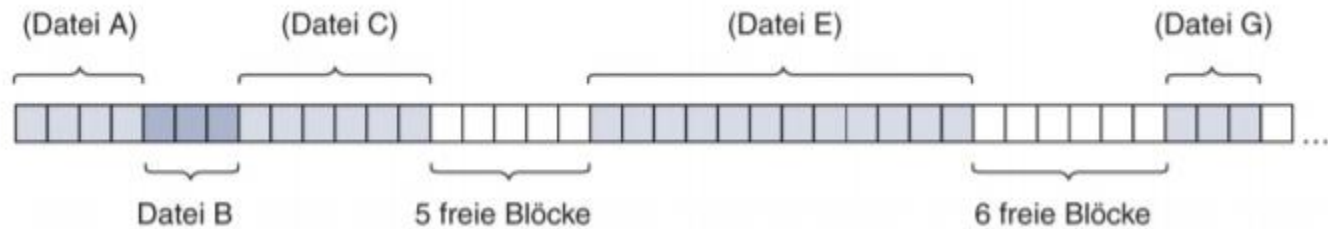
- Dateien bestehen aus mehreren Blöcken
- Das DS muss verwalten, welche Blöcke zu welchen Dateien gehören bzw. welche Dateien sich in den entsprechenden Verzeichnissen befinden
- Dafür gibt es **Inhaltsverzeichnisse** oder auch **Verzeichniseinträge** genannt
- Es gibt unterschiedliche Implementierungen wie Dateien bzw. Verzeichniseinträge abgespeichert und verwaltet werden:
 - Kontinuierliche Speicherung (contiguous allocation)
 - Verkettete Speicherung (linked allocation)
 - Indizierte Speicherung (indexed allocation)

Kontinuierliche Speicherung



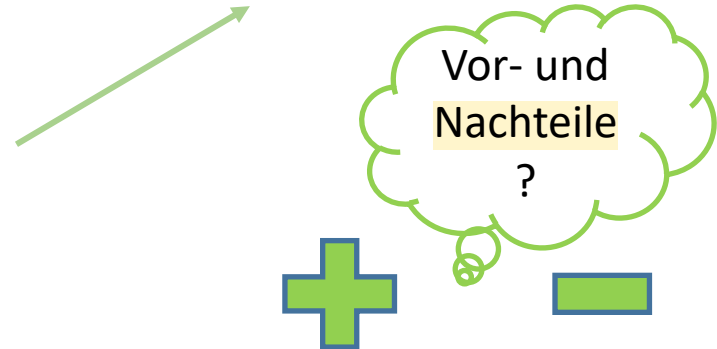
Kontinuierliche Speicherung

- Dateien benötigen oft mehr als nur einen Datenblock:



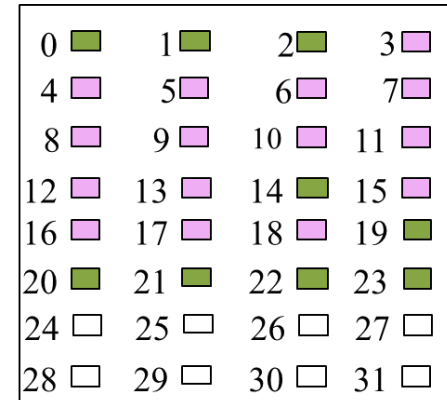
Verzeichnis

Filename	Startblock	Länge
Datei A	0	4
Datei B	4	3
Datei C	7	6
...	...	...



Kontinuierliche Speicherung mit Extents

- Unterteilen einer Datei in Folge von Extents
- Extents werden kontinuierlich gespeichert
- Pro Datei muss Startblock und Länge jedes einzelnen Extents gespeichert werden
- Alle Extents einer Datei werden als Runlist bezeichnet

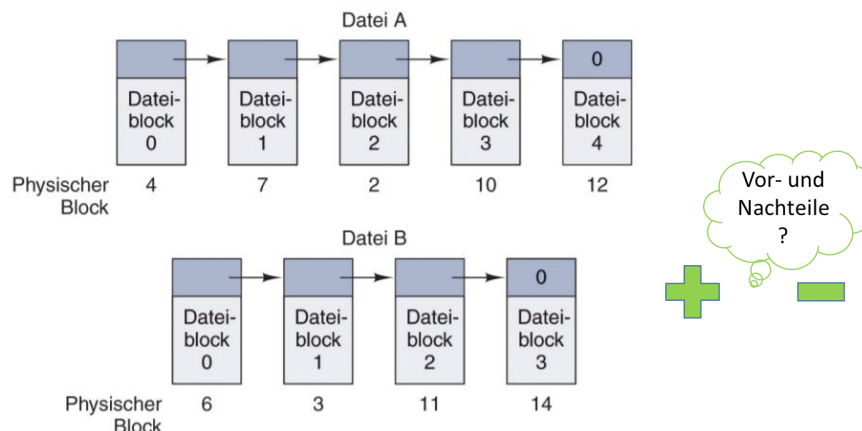
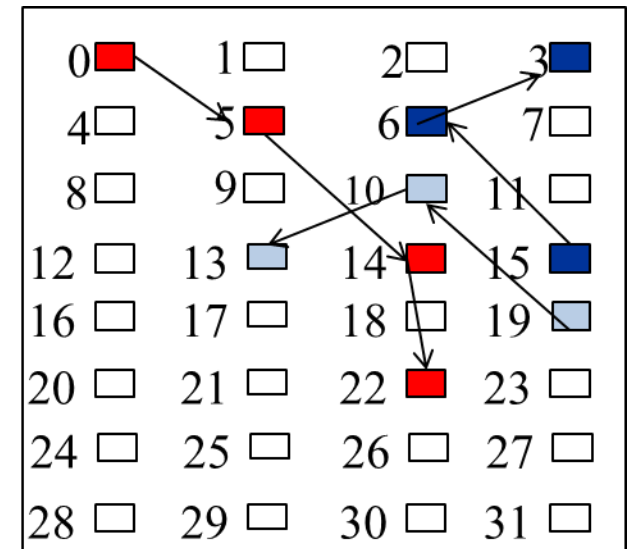


Filename	Startblock	Länge	Runlist
Test.doc	0	3	
	14	1	
	19	5	Runlist
Doku.dat	3	11	
	15	4	

Verzeichnis

Verkettete Speicherung

- Beispiel für Blockgröße 512 Bytes:
 - Die ersten oder die letzten vier Bytes im Block bezeichnen die Blocknummer des nächsten Blocks
 - 508 Bytes Nutzdaten + 4 Byte Zeiger auf nächsten Block



Filename	Startblock	Endblock
file1	0	22
text	15	3
list.dat	19	13

Verzeichnis

Verkettete Liste mit FAT

- Verkettung aller Dateien wird in speziellen Blöcken ausgelagert – in eine sogenannte File Allocation Table (FAT)

0		1		2		3	
4		5		6		7	
8		9		10		11	
12		13		14		15	
16		17		18		19	
20		21		22		23	
24		25		26		27	
28		29		30		31	

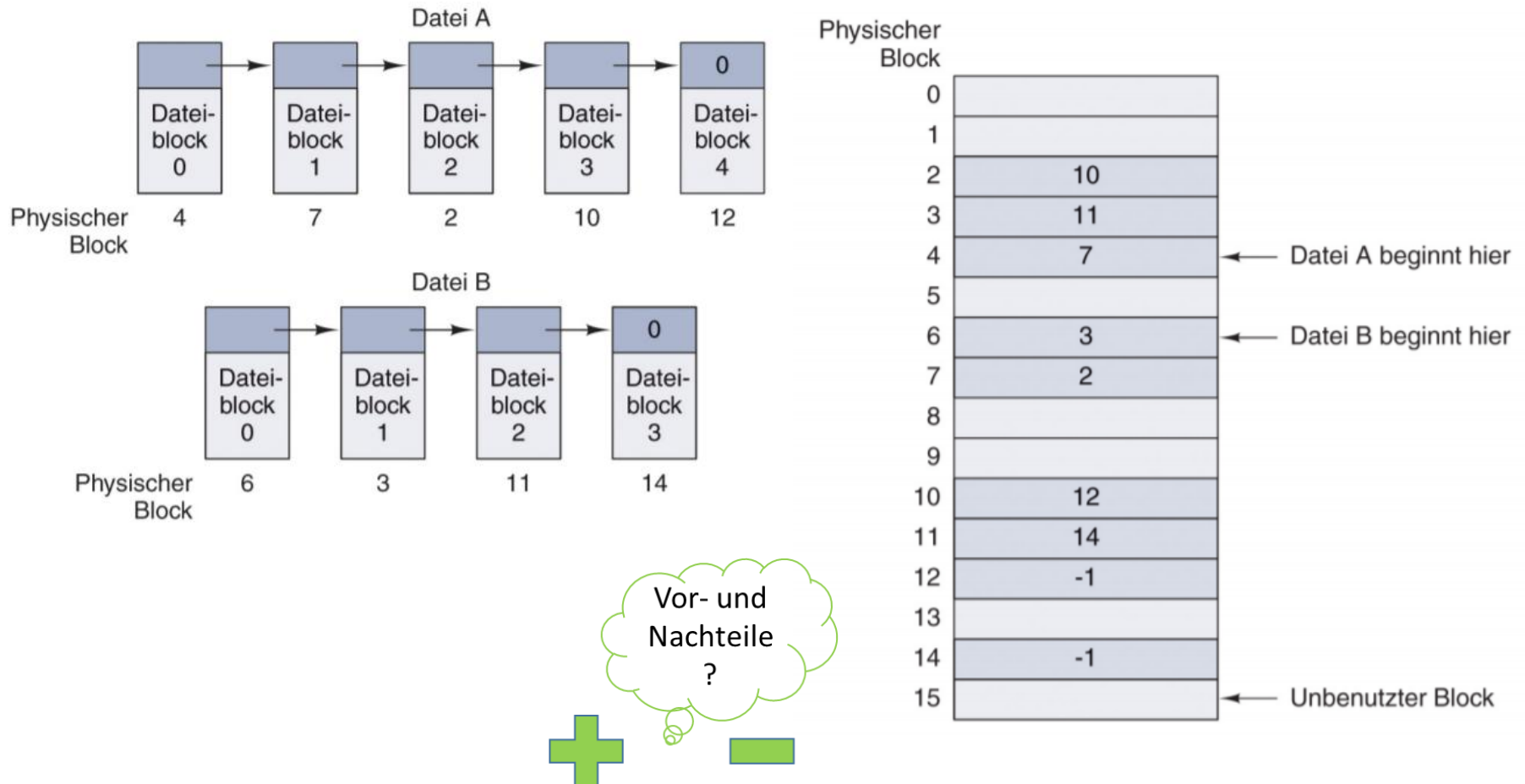
Filename	Startblock
file1	0
text	15
list.dat	19

Verzeichnis

00	05
01	
02	
03	En
04	
05	14
06	03
07	
08	
09	
10	13
11	
12	
13	En
14	22
15	06

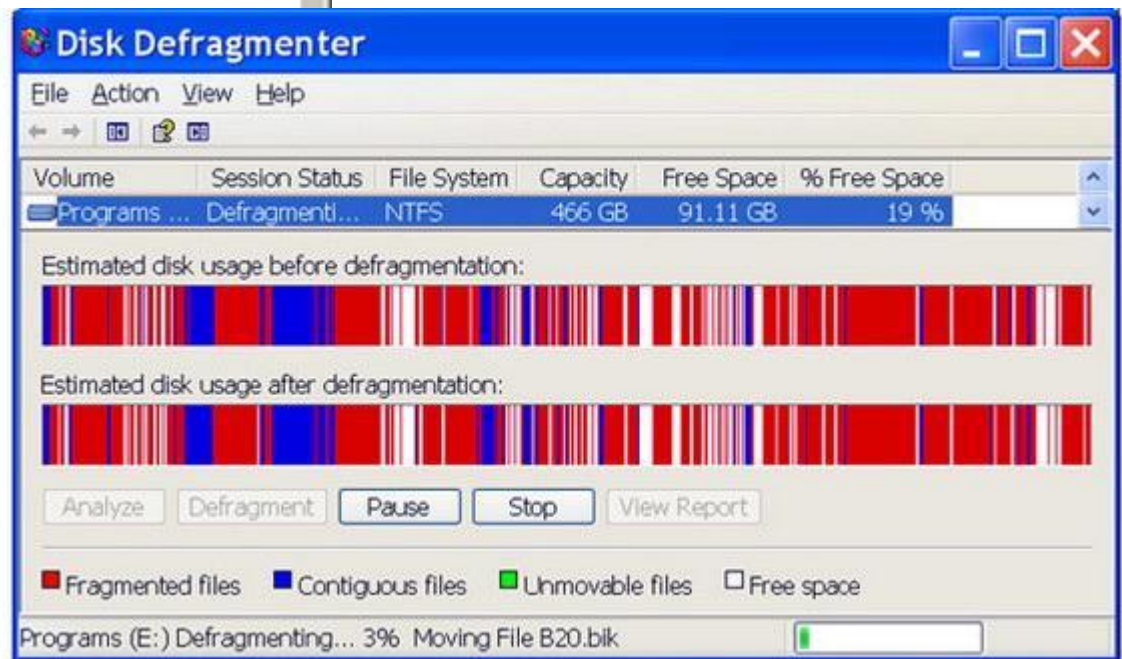
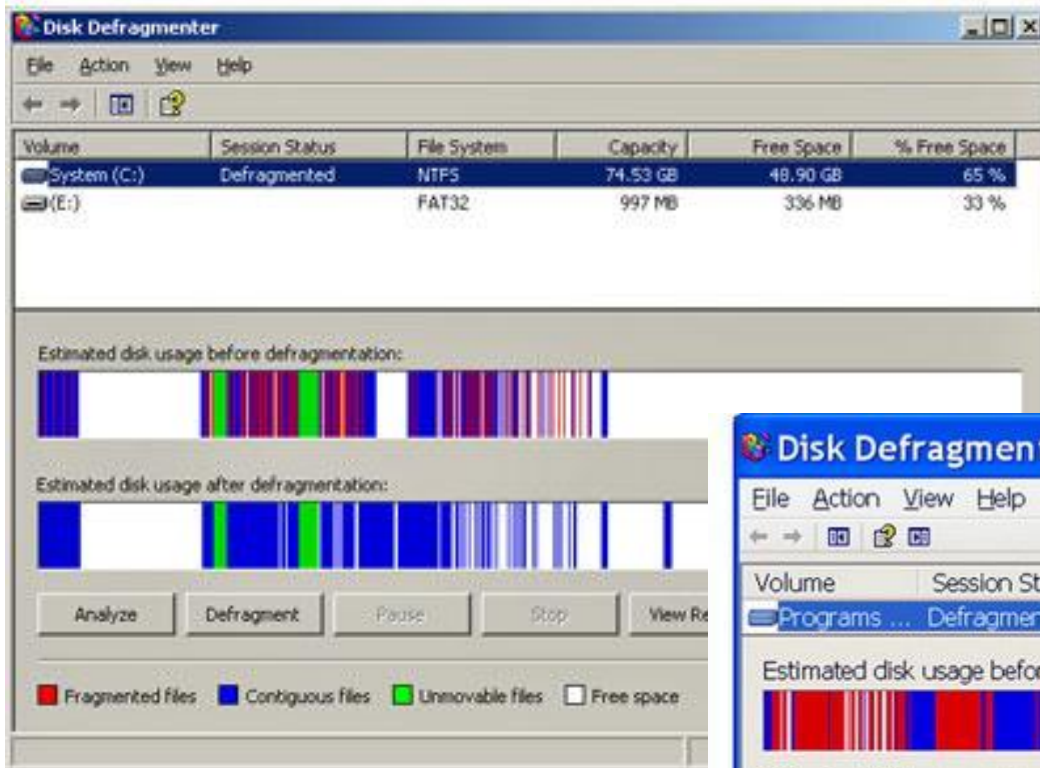
16	
17	
18	
19	10
20	
21	
22	En
23	
24	
25	
26	
27	
28	
29	
30	
31	

Gegenüberstellung verkettete Liste: mit und ohne Tabelle



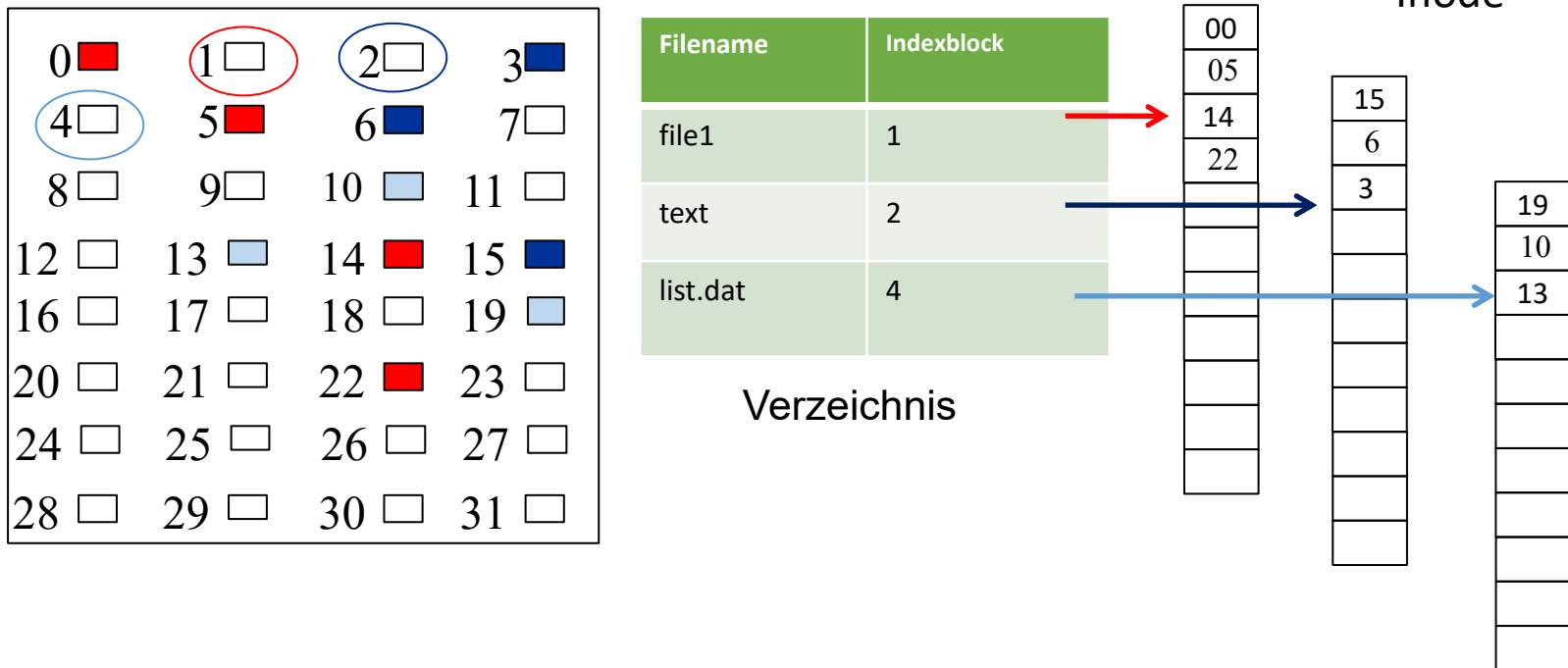
Auszug aus Tanenbaum et. al

Defragmentierung

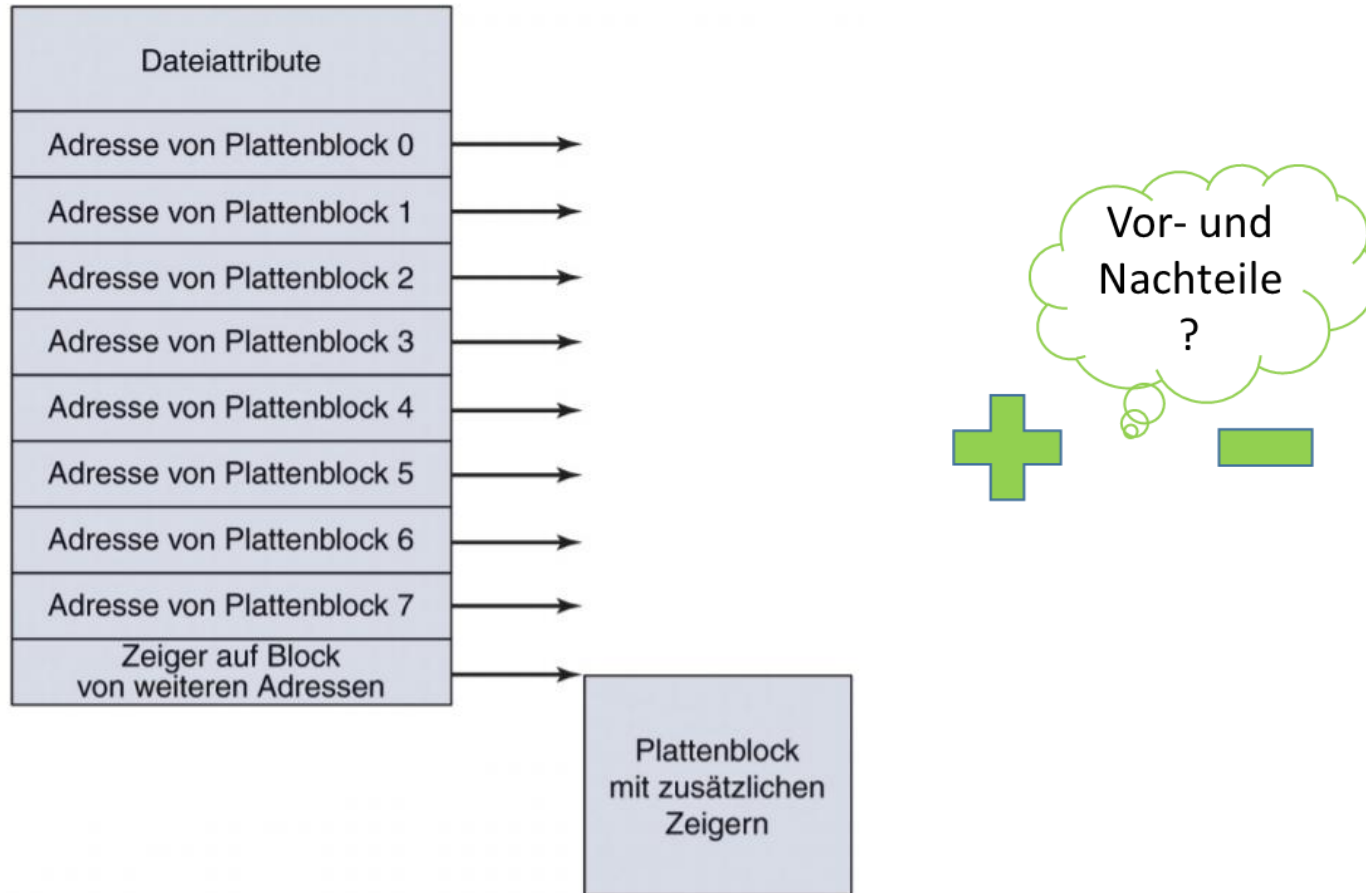


Indizierte Speicherung

- Eigener Block für jede Datei und jedes Verzeichnis, der die LBAs auf die Nutzdaten enthält



Nutzdaten: indizierte Speicherung



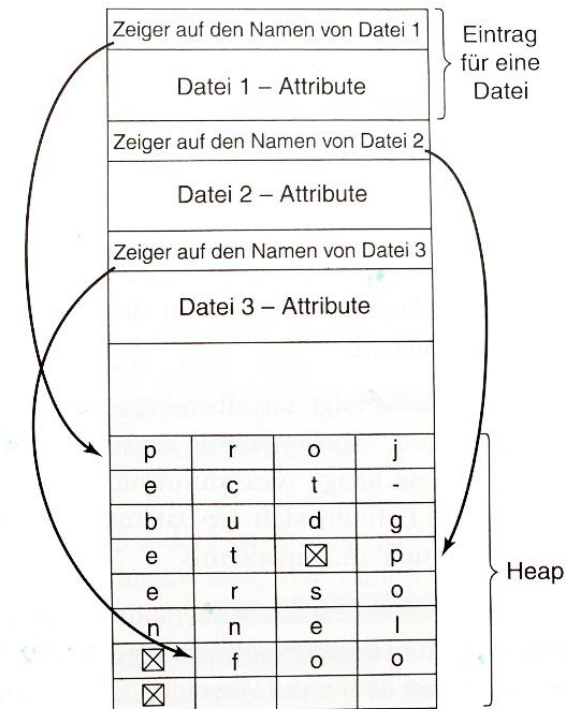
Implementierung von Verzeichnissen

- Für das Öffnen einer Datei benutzt das OS den angegebenen Pfadnamen um den Verzeichniseintrag auf der Platte zu finden
- Verzeichniseintrag stellt Information dar, die für das Auffinden der Plattenblöcke benötigt wird
- Je nach System können das die LBAs der gesamten Datei sein, die Nummer des ersten Blocks, oder die Nummer des Inodes

Implementierung von Verzeichnissen

- 1. Möglichkeit: Bestimmung einer maximalen Länge
 - Problem?
- Alternative: jeder Verzeichniseintrag enthält eine feste Menge
- 2 Möglichkeiten, mit langen Dateinamen in Verzeichnissen umzugehen:
 - linear oder
 - mit einem Heap (Arbeitsspeicher)

Datei 1 – Eintragslänge			
Datei 1 – Attribute			
p	r	o	j
e	c	t	-
b	u	d	g
e	t	☒	
Datei 2 – Eintragslänge			
Datei 2 – Attribute			
p	e	r	s
o	n	n	e
l	☒		
Datei 3 – Eintragslänge			
Datei 3 – Attribute			
f	o	o	☒
:			



Linux ext4 Dateisystem

Implementierungen & Beispiele I

- Konkrete Implementierung von Filesystemen
- **Windows:**
 - Microsoft FAT
 - FAT12, FAT16, FAT32, exFAT
 - Microsoft NTFS
 - Alle Informationen zu Dateien in einer Master File Table (MFT) gespeichert
 - Freispeicherverwaltung: Bitmaps
 - Indizierte Speicherung – Knoten sind MFT-Dateisätze

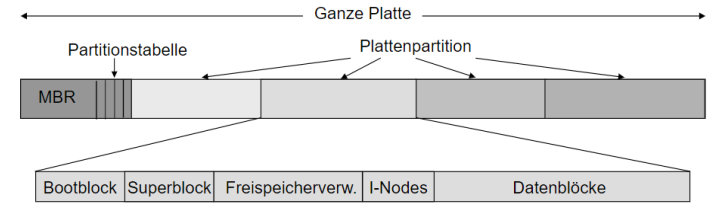
Implementierungen & Beispiele II

- **Unix/Linux/*BSD/Solaris:**
 - ext2, ext3, ext4, UFS, ZFS
 - Unix/Linux – Inode-Liste, arbeitet mit einer abgewandelten Form von Indizierung (Inode-basiertes Dateisystem)
- **macOS:**
 - APFS, (alt: HFS+)
 - Inode-basiertes Dateisystem, ist optimiert für SSD, Flashspeicher
 - Wird auch bei iPhones, iPad
 - Implementierung von Verzeichniseinträgen: Baumstruktur um schnelle Suchen zu ermöglichen

Beispiel Linux: Dateisysteme

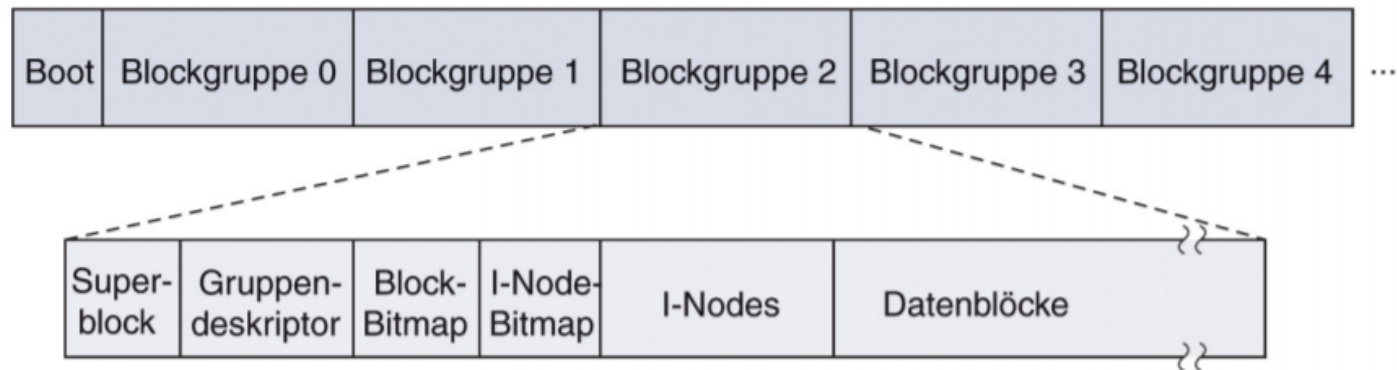
- ext: Extended-Dateisystem
 - Erstes Dateisystem, Schicht für mehrere Dateisystem wurde eingeführt VFS
 - Ext3 wurde Ext2 um Journaling Funktionalität ergänzt, Datenstrukturen zur Partitionsverwaltung blieben unverändert
- ext4 stellt 48 Bit (früher 32 Bit) für die LBA Adressierung zur Verfügung und arbeitet mit Extents
 - Verwaltungsdaten werden über gesamte Partition verteilt
 - Man versucht Verwaltungsdaten und Nutzdaten räumlich zusammenzufassen

Unix-Style Layout Dateisystem



- Bootblock konnte einen Bootloader enthalten
- Superblock
 - Schlüsselparameter des Dateisystems (Name des Dateisystemtyps, Schutzmarkierungen)
- Danach kommt Freispeicherverwaltung
 - Information über die freien Blöcke im Dateisystem
- Inodes
 - Liste von Datenstrukturen → Inhaltsverzeichnis
- Abschluss bilden die Verzeichnisse und Dateien = Datenblöcke

Beispiel Linux Plattenlayout



- Partition ist in Gruppen von Blöcken aufgeteilt
- Der erste Block in einer Gruppe ist der Superblock mit Infos über das Layout des Dateisystems (wie #Inodes, #Plattenblöcke, Zeiger auf die Liste der freien Blöcke)
- Gruppendeskriptor enthält die Infos über die Position der Bitmaps für Inodes und freie Blöcke in der Gruppe und die Anzahl der Verzeichnisse
- Dateien und Verzeichnisse werden nach Möglichkeit innerhalb einer Gruppe gespeichert (Wieso?)

Linux: Inodes

- Inode List

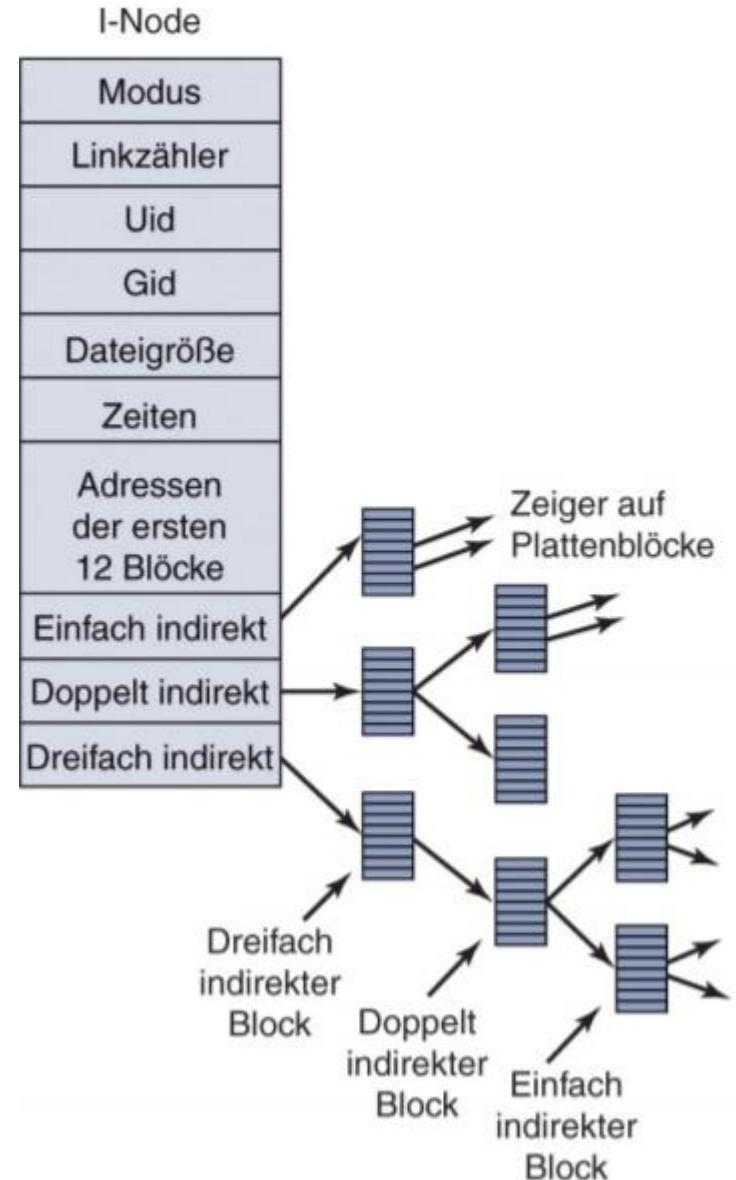
- Die Inode List enthält die Inodes
- Entspricht dem Inhaltsverzeichnis des Dateisystems

- Inode

- Für jedes Objekt (Datei, Verzeichnis,..) wird ein eigener Inode angelegt
 - Datenstruktur, in der die Attribute des Objekts und die LBAs der Nutzdaten gespeichert werden
- Größe Inode == Blockgröße

Datenstruktur Inode

- Jeder Inode beginnt mit Metainformation der Datei/Verzeichnis
- 12 direkte Pointer auf Datenblöcke
- 3 weitere indirekte Pointer:
 - Indirekter Block
 - Doppelt indirekte Blöcke
 - Triple indirekte Blöcke



Linux: Beispiel 1 für maximale Dateigröße

- Annahmen:
 - Größe eines logischen Blocks: 1 KB (2 Sektoren)
 - Adresslänge LBA: 32 Bit (4 Byte)
 - – > 256 LBA's in einem Block
- dann:
 - 12 direkt adressierte Datenblöcke á 1 KB: 12 KB
Dateigröße
 - 1 indirekter Block mit 256 LBA's zu Datenblöcken: 256 KB + 12 KB = max 268 KB Dateigröße
 - 1 doppelt indirekter Block: ? Dateigröße
 - 1 dreifach indirekter Block: ? Dateigröße

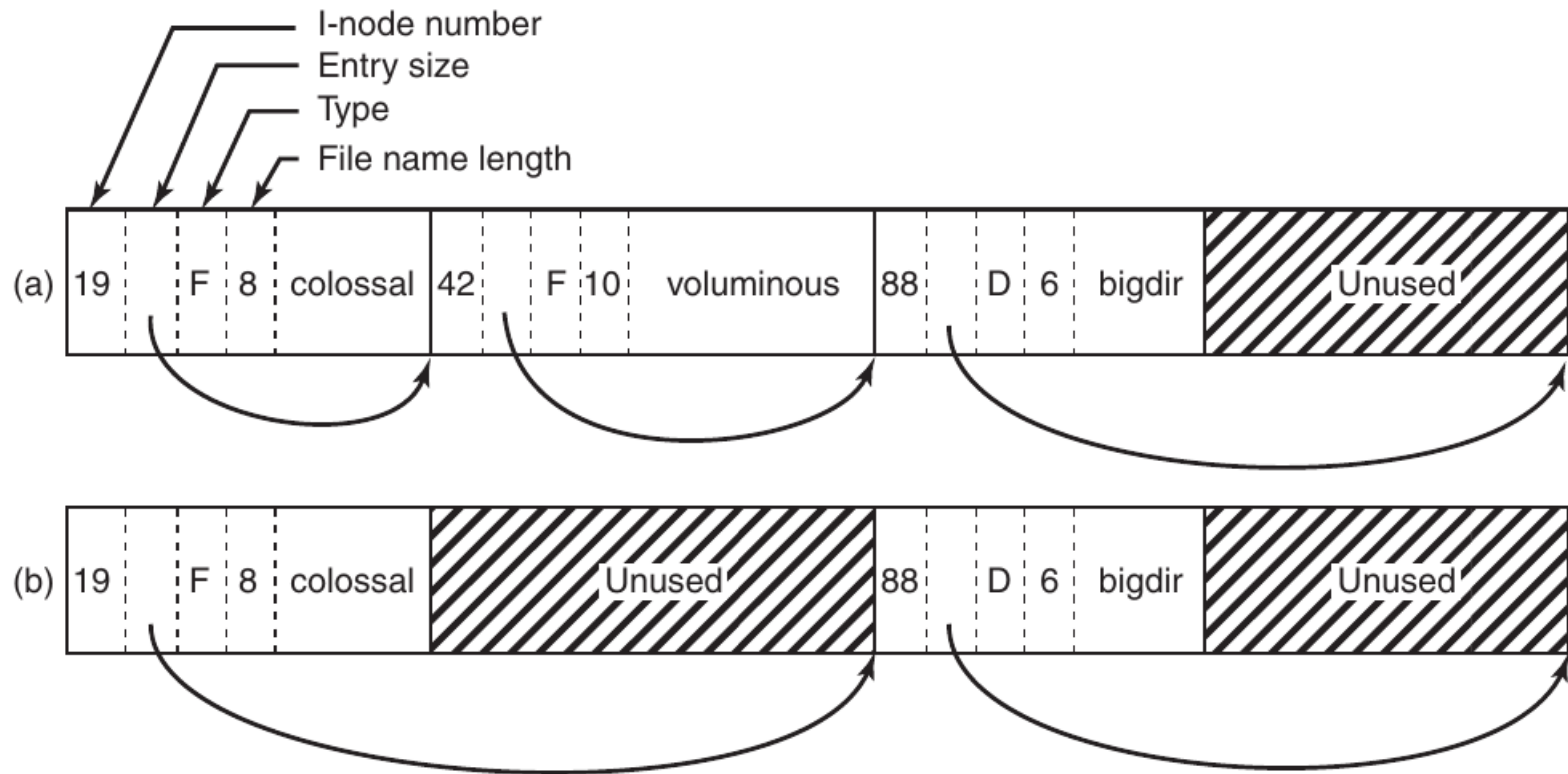
Linux: Beispiel 2 für maximale Dateigröße

- Annahmen:
 - Größe eines logischen Blocks: 4 KB (8 Sektoren)
 - Adresslänge LBA: 32 Bit (4 Byte)
 - – > 1024 LBA's in einem Block
- dann:
 - 12 direkt adressierte Datenblöcke á 4 KB: 48 KB Dateigröße
 - 1 indirekter Block mit 1024 LBA's zu direkten Datenblöcken: 4096 KB + 48 KB = max 4144 KB Dateigröße
 - 1 doppelt indirekter Block: ? Dateigröße
 - 1 dreifach indirekter Block: ? Dateigröße

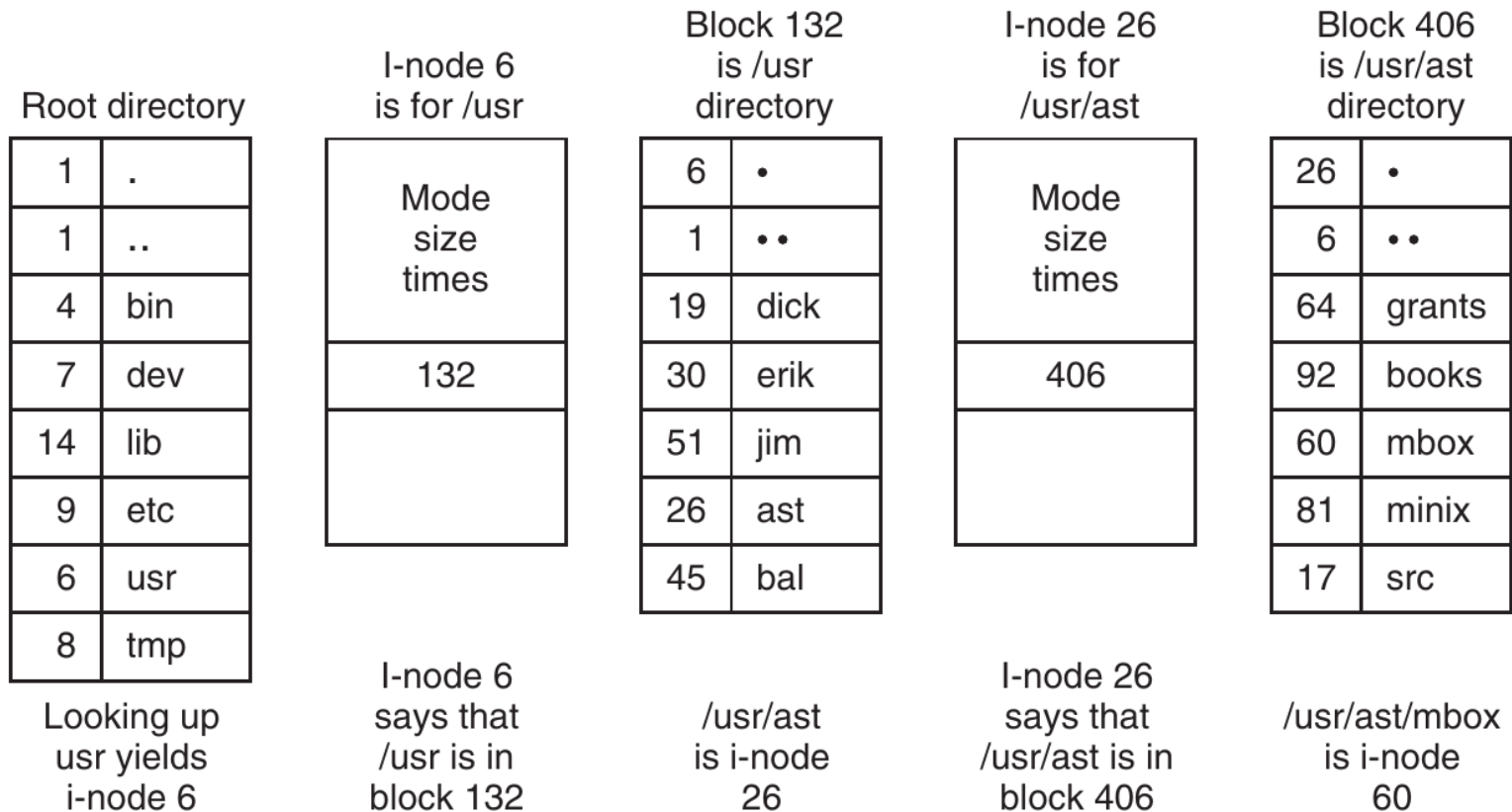
Linux: Implementierung von Verzeichnissen

- Implementierung als verkettete Liste oder mit zusätzlichem Index in Form eines H-trees
- Verkettete Liste (hier beschrieben):
 - Jedes Verzeichnis besteht aus einer Liste sogenannter **directory entries**. Das ist eine Datenstruktur, die für jedes Objekt des Verzeichnisses angelegt wird.
 - Die Liste der directory entries wird bei einer normalen Datei über die LBA's im dazu gehörigen Inode des Verzeichnisses verwaltet.
 - Die Länge eines **directory entries** hängt von der Länge des Filenamens ab.
 - Insgesamt wird in jedem **directory entry** folgendes gespeichert:
 - Inode nummer des Objekts
 - Länge des directory entries
 - Länge des Namens des Objekts
 - Typ des Objekts
 - Name des Objekts

Linux: Aufbau einer Verzeichnisdatei



Linux: Zugriff über Pfadangabe



Beispiel: Zugriff auf `/usr/ast/mbox`

Dateisystem Erweiterungen

Journaling

- Journaling oder wiederherstellbare Dateisysteme
- Jede Modifikation im Dateisystem wird **bevor** sie physikalisch auf der Festplatte ankommt in einer speziellen Datei – einer Logdatei - protokolliert
 - Jede Modifikation wird in Suboperationen zerlegt, dann wird eine Modifikation als **Transaktion** bezeichnet
 - Beispiele für Modifikationen: Anlegen einer neuen Datei, Löschen einer Datei, Schreiben in eine Datei, Löschen von Sätzen einer Datei, etc.
 - Beispiel: Suboperationen beim **Löschen einer Datei**
 - **Markiere Transaktionsbeginn**
 - Freigeben der belegten Datenblöcke durch Setzen der Bits in der Bitmap des Dateisystems (falls Bitmaps verwendet werden)
 - Freigeben des Eintrags der Datei
 - Löschen des Dateieintrags im Verzeichnis
 - **Ende der Transaktion (= Commit)**

Journaling

- Nach einem Systemabsturz startet ein Recovery Utility (z.B.: chkdsk), das auf Grund der Logdatei erkennt, welche Suboperationen einer Transaktionen bis zum Systemabsturz vorgenommen wurden und welche nicht.
- Es *entfernt* alle Suboperationen (**Undo**), wenn **kein Transaktionsende** gefunden wurde.
- Es *komplettiert* alle Suboperationen auf der Festplatte (**Redo**), wenn **das Transaktionsende** gefunden wurde.
- Metadaten-Journaling
 - lediglich die Konsistenz der Metadaten wird garantiert
- Full-Journaling
 - auch die Konsistenz der Nutzdaten gewährleistet

Extents (am Beispiel ext4)

- Bis zu ext4 referenzierten Inodes Blöcke direkt
 - Ab ext4 können zusätzlich Extents referenziert werden
- Extent ist eine Menge an aufeinanderfolgenden Blöcken
 - Bis zu 128 MB möglich
- 4 Extents können direkt von einem Inode referenziert werden
- > 4 Extents werden durch eine Baumstruktur indiziert
- Vorteil
 - Bessere Performance bei großen Dateien
 - Minimierung der Fragmentierung
 - Rückwärtskompatibel mit Ext3

Copy-on-Write (COW) und B+-Trees

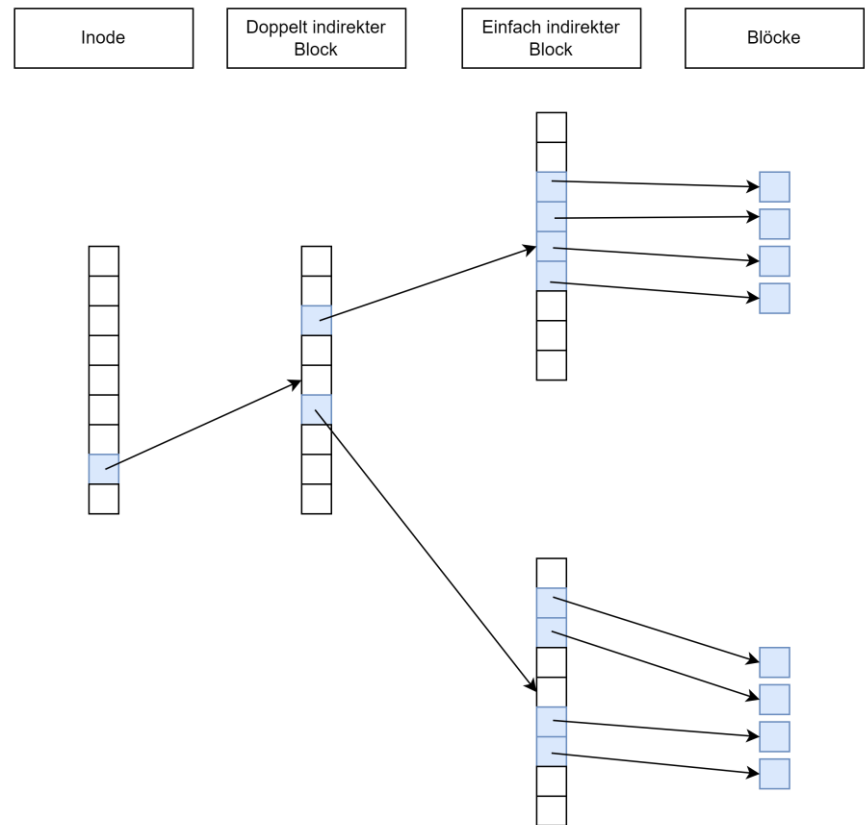
- Technik von modernen DS, z.B. ZFS, Btrfs, ReFS, Bcachefs
- Full Journaling ineffizient
 - COW Dateisysteme unterstützen Daten und Metadatenkonsistenz
 - Zusätzlich erlauben diese Dateisysteme das Erstellen von Snapshots -> effiziente Backups, Versionierung
- COW generell
 - Statt eine Kopie anzufertigen, wird ein Zeiger vergeben
 - Dies reicht für den Lesezugriff
 - Sobald geschrieben wird, wird eine (partielle) Kopie erstellt

Copy-on-Write (COW) und B+-Trees

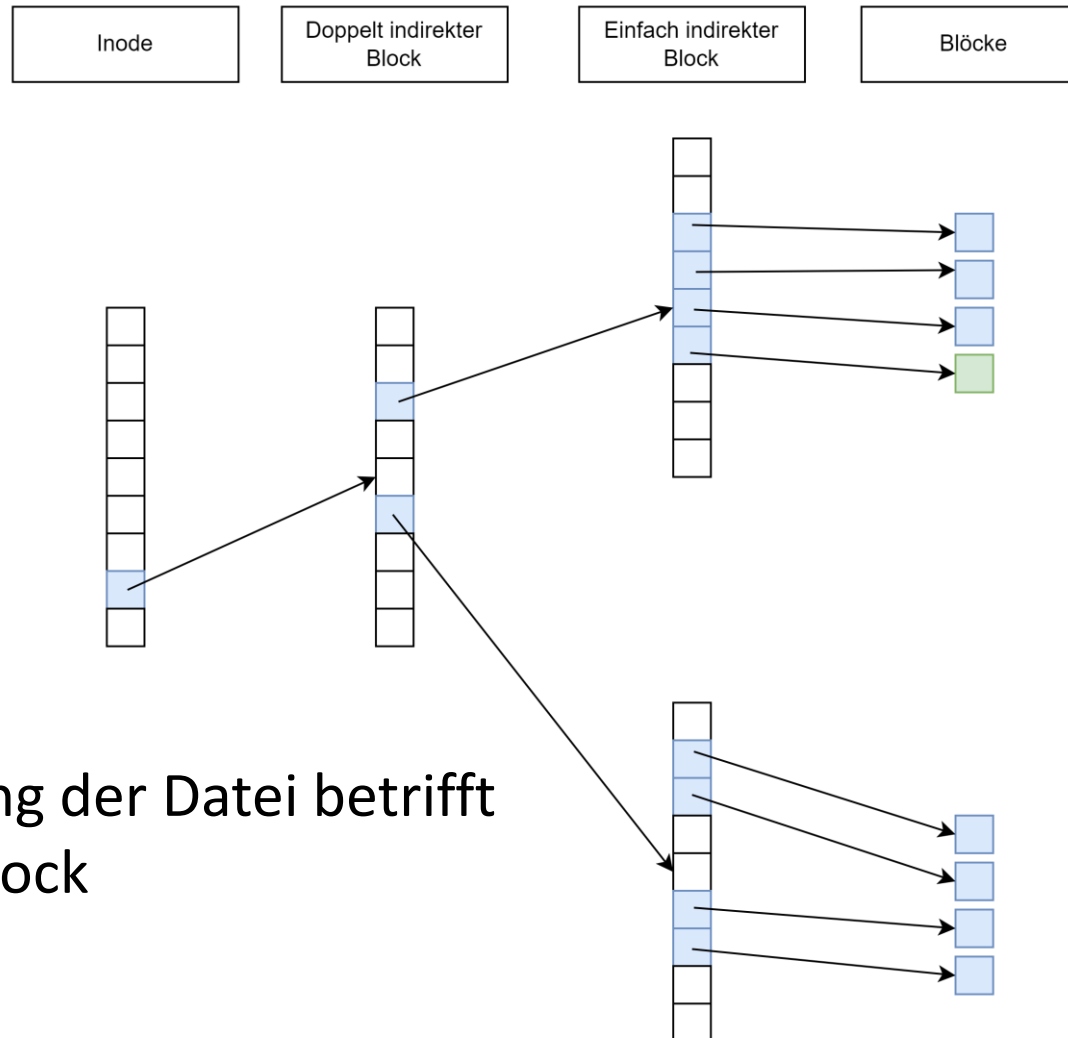
- COW DS überschreiben also keine Daten
 - Alle Änderungen werden auf freie Blöcke geschrieben
 - „Sicher“ im Fall von Crashes
 - Ermöglicht inkrementelle Snapshots
- Zusätzlich Verwendung von B+-Baum Varianten, z.B.
 - Beschleunigung Random Access (z.B. auch in Extents bei ext4) für große Dateien
 - Durchlaufen einer ungeordneten Liste an LBAs ist zu langsam
 - Verzeichnishierarchie kann als B+-Baum dargestellt werden
 - Beschleunigt den Lookup von Dateien in Verzeichnissen

Copy-on-Write (COW) - Beispiel

- Beispiel zeigt grundsätzliche Funktionsweise von COW
- Änderung einer Datei
- Hypothetische, angepasste Inode Datenstruktur für dieses Beispiel

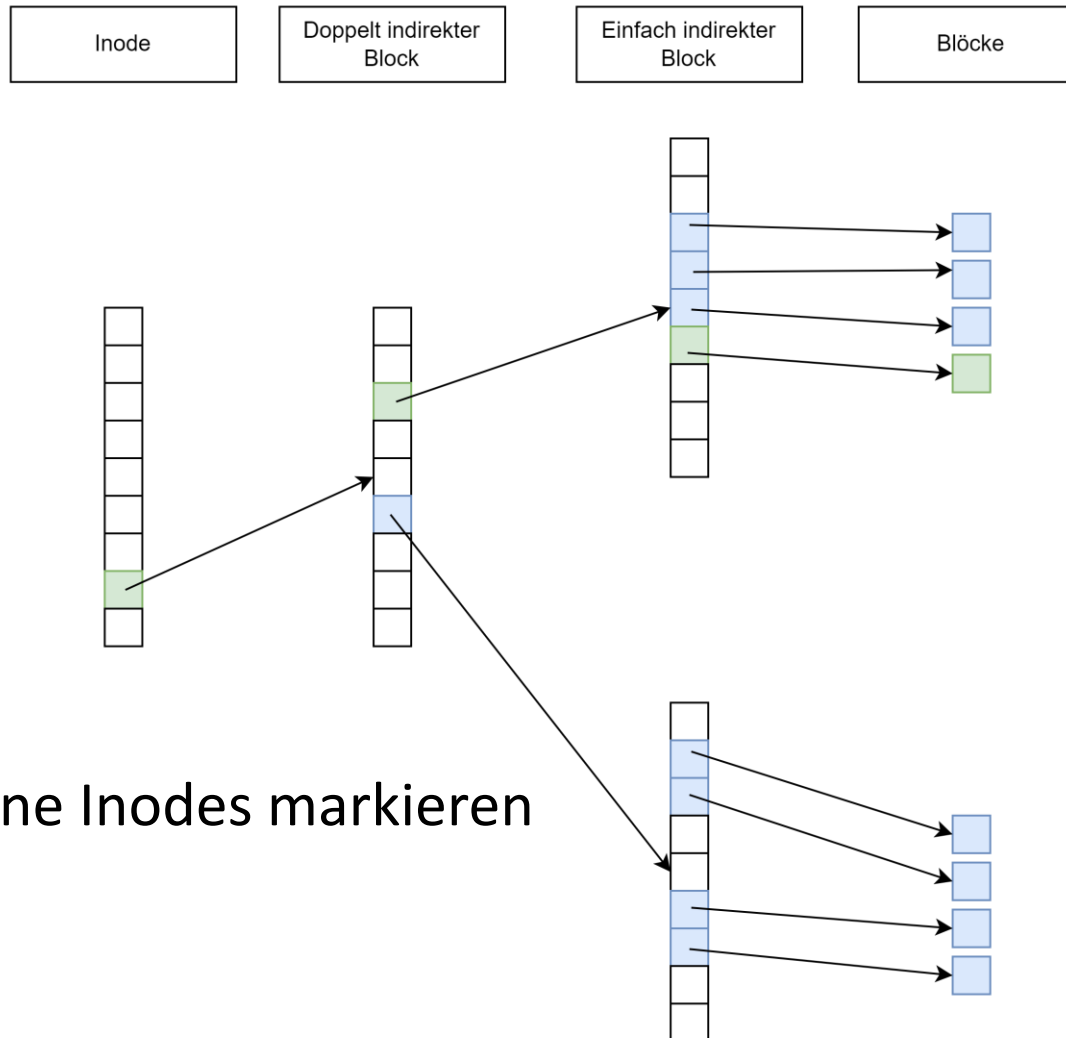


Copy-on-Write (COW) - Beispiel

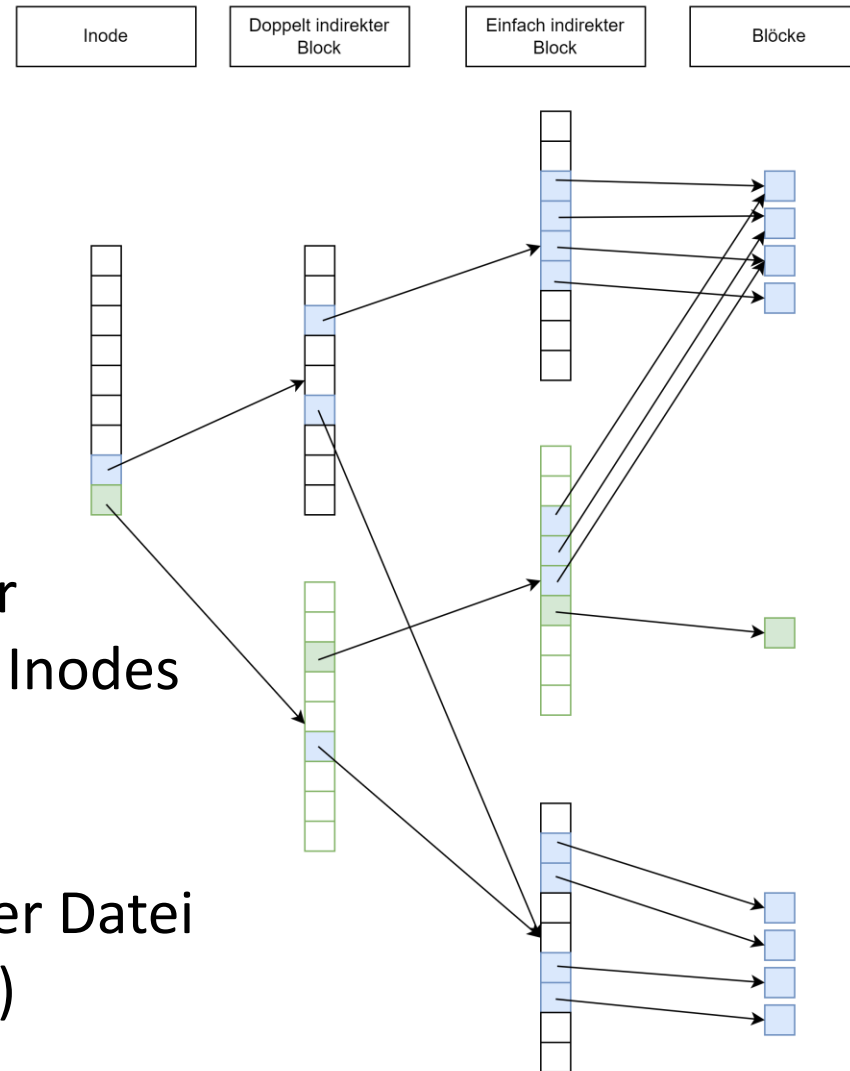


- Änderung der Datei betrifft einen Block

Copy-on-Write (COW) - Beispiel



Copy-on-Write (COW) - Beispiel



- Kopieren der betroffenen Inodes und Blöcke
- Ergebnis: 2 Versionen der Datei (alt und neu)

Zusammenfassung DS

- Partitionen, Adressierung, Sektoren, Blöcke
- Booten eines Computers
- HDD & SSD
- Logische vs. physische Sicht von Dateien und Verzeichnissen
- Layout eines Dateisystems
- Beispiele von Dateisystemen
- Ausgewähltes Beispiel: Linux